Understanding LeanVAE for Beginners

https://gemini.google.com/app/3b48c4a47a776540

User prompt: in this paper what does say this code. let's fine. in deep. like beginner. ------ class PEG3D(nn.Module): def __init__(self, dim): super().__init__() self.ds_conv = nn.Conv3d(in_channels=dim, out_channels=dim, kernel_size=(3,3,3), groups = dim) self.is_first_chunk = True self.causal_cached = None self.tile_inference = False def forward(self, x): x = rearrange(x, 'b t h w d -> b d t h w') if self.tile_inference: if self.is_first_chunk: x = F.pad(x, (1, 1, 1, 1, 2, 0), value=0.) else: x = F.pad(x, (1, 1, 1, 1, 0, 0), value=0.) x = torch.concatenate((self.causal_cached, x), dim=2) self.causal_cached = x[:, :, -2:].clone() else: x = F.pad(x, (1, 1, 1, 1, 2, 0), value=0.) x = self.ds_conv(x.contiguous()) x = rearrange(x, 'b d t h w -> b t h w d') return x

Response: PEG3D implements the **causal 3×3×3 depthwise convolution** (3x3x3 Dconv) used inside LeanVAE's **NAF** and **ResNAF** blocks (discussed in **Section 4.2** and **Appendix D**, Figure 1b and Figure 8b). Its job is to help the model understand local 3D neighborhoods (across time, height, and width) while keeping computation lightweight, preventing the model from "seeing into the future," and allowing long videos to be processed in smaller memory-friendly chunks without visual glitches.

1. Where This Code Fits in the Paper

- **Section 4.2 (Encoder and Decoder):** The paper introduces the **Neighborhood-Aware Feedforward (NAF)** module as the core building block. Inside each NAF block, the video first passes through a 3×3×3 depthwise convolution (self.ds_conv) with causal temporal padding before entering a feedforward network.
- **Appendix D (Tiling Inference & Figure 8b):** Processing a high-resolution or long video all at once can crash your GPU memory. To fix this, you split the video into smaller time chunks (**tiling inference**). This code implements the exact **Caching Strategy** shown in Figure 8(b), saving the last 2 frames of a chunk so the next chunk has smooth temporal context.

2. Key Concepts for Beginners

- **3D Convolution (Conv3d):** A standard 2D convolution slides a 3×3 window over an image's height and width. A 3D convolution slides a 3×3×3 cube over **Time (*t*), Height (*h*), and Width (*w*)** to capture both spatial details and motion across frames.
- **Depthwise Convolution (groups = dim):** Normally, a convolution mixes all feature channels together, which requires heavy math. Setting groups = dim gives each channel its own dedicated 3×3×3 filter. It looks at neighboring pixels and frames without mixing channels, making the model drastically faster.
- **Causal Padding:** A 3×3×3 cube normally looks at 1 past frame, the current frame, and 1 future frame. In a **causal** setup, the model is forbidden from looking at future frames (each frame only interacts with previous frames). Instead, it looks at **2 past frames** and the **current frame** (2+1=3).
- **Padding (F.pad):** When a 3×3×3 cube slides over the edges of a video, it shrinks the output size by 2

along each dimension unless you add extra border pixels ("padding"). In PyTorch, F.pad(x, (left, right, top, bottom, front_time, back_time)) pads the dimensions from last to first (Width, then Height, then Time).

3. Line-by-Line Code Breakdown

The Setup (__init__)

- self.ds_conv = nn.Conv3d(in_channels=dim, out_channels=dim, kernel_size=(3,3,3), groups = dim): Creates the 3×3×3 depthwise convolution layer. Because no padding is set inside Conv3d, the code handles padding manually in forward.
- self.is_first_chunk = True: Tracks whether the model is currently looking at the very first segment of a split video.
- self.causal_cached = None: An empty storage slot that will hold the last 2 frames of the current video chunk to pass over to the next chunk.
- self.tile_inference = False: A toggle switch. When False, the whole video is processed at once. When True, the video is processed piece-by-piece in chunks.

Processing the Video (forward)

- x = rearrange(x, 'b t h w d -> b d t h w'): Reshapes the tensor. Transformer-style layers usually keep channels (d) at the end: (Batch, Time, Height, Width, Channels). PyTorch's Conv3d requires channels right after Batch: (Batch, Channels, Time, Height, Width).
- if self.tile_inference: Checks if chunk-by-chunk mode is turned on.
- if self.is_first_chunk: x = F.pad(x, (1, 1, 1, 1, 2, 0), value=0.): For the very first chunk, there are no previous frames in memory. It pads Width by (1, 1) (left/right), Height by (1, 1) (top/bottom), and Time by (2, 0)—adding **2 blank zero-frames at the beginning** and **0 frames at the end**. This forces the 3-frame window to only look backward in time.
- else: x = F.pad(x, (1, 1, 1, 1, 0, 0), value=0.): For the second chunk onward, we only pad Width (1, 1) and Height (1, 1). We add (0, 0) to Time because we are about to attach real frames from the previous chunk instead of zeros.
- x = torch.concatenate((self.causal_cached, x), dim=2): Glues the 2 saved frames from the end of the previous chunk (self.causal_cached) onto the front of the current chunk along the Time dimension (dim=2). This matches **Figure 8(b)** in the paper, where frames 3 and 4 are cached in front of frame 5.
- self.causal_cached = x[:, :, -2:].clone(): Grabs the **last 2 frames** (-2:) of the current padded tensor and saves a copy in memory so the *next* chunk can use them.
- else: x = F.pad(x, (1, 1, 1, 1, 2, 0), value=0.): If tile_inference is False (processing the entire video at once), it simply pads Width (1, 1), Height (1, 1), and puts 2 zero-frames at the front of Time (2, 0).
- x = self.ds_conv(x.contiguous()): .contiguous() organizes the tensor cleanly in computer memory after all the rearranging and concatenating, and self.ds_conv runs the 3×3×3 depthwise convolution. Because we added 2 frames to Time, 2 pixels to Height, and 2 pixels to Width, the 3×3×3 filter shrinks the tensor right back to its exact original (t, h, w) size.
- x = rearrange(x, 'b d t h w -> b t h w d'): Moves the channel dimension (d) back to the end so the tensor is ready for the FeedForward network that follows in the NAF block.

Would you like to walk through a concrete numerical example with tensor shapes (e.g., watching how a 5-frame video chunk changes shape at each line), or look at how this connects to the rest of the ResNAF block?

User prompt: what does meaning of groups in conv3d

Response: The groups parameter in PyTorch's Conv3d controls how input channels are connected to output channels during the convolution process.

Standard Convolution (groups=1) By default, a 3D convolution filter looks at every single input channel to produce one output channel. If a video has 128 input channels, the mathematical filter mixes the information from all 128 channels together simultaneously. This cross-channel mixing requires massive computational power.

Grouped Convolution (groups > 1) Setting groups greater than 1 splits the input and output channels into isolated chunks. If you have 128 channels and set groups=4, the convolution divides the channels into 4 separate blocks of 32. The filters in the first block only look at the first 32 input channels, and so on.

Depthwise Convolution (groups = channels) In the PEG3D code you provided, the parameter is set to groups = dim (where dim is the total number of input channels). This creates a **depthwise convolution**. Instead of mixing channels, the network assigns one dedicated 3×3×3 filter to each individual channel. The filter slides across time, height, and width for that specific channel alone, learning local spatial and temporal patterns independently. This isolates spatial learning from channel mixing, drastically reducing the number of parameters and making the model significantly faster to train.

User prompt: in padding value=0. -> means.

Response: value=0. tells the padding function to fill the newly created space with the exact number **zero** (specifically, a floating-point 0.0). This is commonly called "zero-padding."

In the context of this video model:

- **Time Padding:** When the code adds 2 frames to the beginning of the chunk, value=0. means it is inserting 2 completely blank "zero-frames."
- **Spatial Padding:** When the code adds 1 pixel to the top, bottom, left, and right, it is wrapping each frame in a 1-pixel border of zeros.

Why use zero? In neural networks, zero is mathematically neutral. When the convolution filter slides over the outer edges of the video, it multiplies its learned weights against the pixel values. Since multiplying by zero equals zero, these artificial padded areas do not activate the filter. This ensures the model only learns from the real video content and doesn't get confused by fake data at the boundaries.

---

*User prompt: let's teach me this function. ------ class GEGLU(nn.Module): def forward(self, x): x, gate = x.chunk(2, dim=-1) return F.gelu(gate) * x*

Response: GEGLU (Gated Linear Unit with GELU) acts as a multiplicative filter where one half of your feature channels dictates how much of the other half is allowed to pass through.

x, gate = x.chunk(2, dim=-1) This splits the input tensor exactly in half along the final dimension, which in PyTorch is typically the channel or feature dimension. If a tensor arrives with 512 channels, x retains the first 256, and gate takes the remaining 256.

return F.gelu(gate) * x The network applies the non-linear Gaussian Error Linear Unit (GELU) exclusively to the gate tensor. It then performs an element-wise multiplication between this activated gate and the raw, unactivated x tensor.

Instead of applying a static activation function across the entire representation, this creates a dynamic, data-dependent routing mechanism. The model actively learns to use the gate vector to scale, modulate, or suppress the signal in the x vector. This gating behavior yields smoother gradient norms and a richer representation capacity, making it a highly effective replacement for standard feedforward layers when optimizing training dynamics in generative architectures.

---

*User prompt: let's teach me in deep, as beginner. ------ def ffd(dim, mult=4, dropout=0.): inner_dim = int(mult * (2 / 3) * dim) return nn.Sequential(nn.LayerNorm(dim), nn.Linear(dim, inner_dim * 2, bias=False), GEGLU(), nn.Dropout(dropout), nn.Linear(inner_dim, dim, bias=False))*

Response: The ffd function builds the **FeedForward** network used inside LeanVAE's **NAF** and **ResNAF** blocks (shown in **Figure 1b** and **Section 4.2**).

To understand why this exists, remember how PEG3D worked: it used a depthwise convolution (groups=dim), where each channel looked at its own spatial and temporal neighbors, but **never talked to other channels**. The ffd function does the exact opposite: it looks at a single pixel/frame position at a time and **mixes all the channels together**, allowing the network to combine different features.

1. Function Inputs (def ffd(dim, mult=4, dropout=0.):)

- **dim:** The number of channels entering and leaving this block (for example, 512 in the main encoder/decoder).
- **mult=4:** The expansion multiplier. Feedforward networks work by temporarily expanding the number of channels into a wider space so the model can untangle complex patterns, then compressing them back down to dim. Normally, a multiplier of 4 means expanding dim to 4 * dim.
- **dropout=0.:** The percentage of values to randomly set to zero during training to prevent overfitting. By default, it is 0. (turned off).

2. The "2/3" Math Trick (inner_dim = int(mult * (2 / 3) * dim)) If a standard feedforward network expands dim by 4x, it uses two linear layers:

1. Expand: dim → 4 * dim (uses $4\times dim^2$ weights)
2. Shrink: 4 * dim → dim (uses $4\times dim^2$ weights) *Total weights = $8\times dim^2$.*

However, because this network uses **GEGLU()**, which cuts the channel dimension in half, the first linear layer has to output **double** the channels (inner_dim * 2). If we didn't shrink inner_dim, our first layer would output 8 * dim channels, making the network much heavier and slower!

By scaling mult by **2 / 3**, inner_dim becomes $\frac{8}{3}$dim (about 2.67×dim):

1. Expand (doubled for GEGLU): dim → 2 * (8/3 * dim) = $_{16}$dim$_2$ weights
2. Shrink (after GEGLU cuts it in half): (8/3 * dim) → dim = $_{38}$dim$_2$ weights *Total weights* = $_{16}$+$_{38}$=$_{24}$=*8×dim*$_2$. This 2 / 3 trick gives you the power of GEGLU while keeping the **exact same parameter count and compute cost** as a standard 4x feedforward layer.

3. Step-by-Step Pipeline (nn.Sequential(...)) nn.Sequential chains operations together so data flows through them in order. Let's trace a tensor with dim = 300 channels (chosen so the 2/3 math is clean: inner_dim = int(4 * (2/3) * 300) = 800):

1. **nn.LayerNorm(dim) (Channel Count: 300 → 300):** Stabilizes the numbers across the 300 channels for each individual token so they have a balanced mean and variance before heavy math happens. (Note: In Section 4.2 & 5.4, the authors avoid LayerNorm on raw image patches at the very start of the network to prevent grid artifacts, but using it here deep inside the feature backbone is standard).
2. **nn.Linear(dim, inner_dim * 2, bias=False) (Channel Count: 300 → 1600):** Mixes all 300 channels together and projects them up to 1600 (800 * 2) channels. bias=False skips adding a constant offset number, which saves memory and works smoothly with LayerNorm and GEGLU.
3. **GEGLU() (Channel Count: 1600 → 800):** Splits the 1600 channels into two groups of 800 (x and gate), applies GELU to gate, and multiplies them together (F.gelu(gate) * x). Now we have 800 filtered, high-value channels.
4. **nn.Dropout(dropout) (Channel Count: 800 → 800):** If dropout > 0, randomly zeroes out some values during training. Since dropout=0. by default, data passes right through unchanged.
5. **nn.Linear(inner_dim, dim, bias=False) (Channel Count: 800 → 300):** Mixes the 800 gated channels and compresses them back down to the original 300 (dim) channels, ready to be added to the residual connection in the ResNAF block.

Would you like to look at how PEG3D and ffd combine inside the full NAF or ResNAF class next?

---

*User prompt: let's teach me in deep, as beginner ------ def ffd(dim, mult=4, dropout=0.): inner_dim = int(mult * (2 / 3) * dim) return nn.Sequential(nn.LayerNorm(dim), nn.Linear(dim, inner_dim * 2, bias=False), GEGLU(), nn.Dropout(dropout), nn.Linear(inner_dim, dim, bias=False))*

Response: Think of the ffd (FeedForward) function as a conceptual processing facility that refines raw video features into complex concepts. In the LeanVAE architecture, this block immediately follows the depthwise 3D convolution (PEG3D). While the convolution looks at neighboring pixels across space and time without mixing channels, ffd does the exact opposite: it looks at one specific pixel and mixes all of its channels together to build higher-level representations.

Here is what happens to a single data point (like one pixel across 512 channels) as it travels through the nn.Sequential pipeline:

1. nn.LayerNorm(dim): The Reset Button If some feature channels have massively high values (like 1,000) and others are tiny (like 0.01), the network's mathematical gradients become unstable. LayerNorm calculates the average and variance of the channels for that specific pixel, scaling them so they are neatly balanced and centered. It equalizes the mathematical "volume" before the heavy transformations begin.

2. nn.Linear(dim, inner_dim * 2, bias=False): The Expansion Phase A Linear layer is a web of multiplications. It takes the original channels and mixes them together to create a massive number of brand-new, expanded features. For example, it might combine a "redness" channel and a "motion" channel

to create a new feature representing "fast-moving red object." By drastically expanding the channel count (inner_dim * 2), the network gets a wide, temporary workspace to generate every possible combination of features it might need for reconstruction.

3. GEGLU(): The Editor As explored in the previous breakdown, GEGLU splits that massive expanded workspace in half. It uses one half as a "gate" to evaluate the other half. If the "fast-moving red object" feature isn't helpful for this specific video, GEGLU multiplies it by zero (or a tiny fraction) to suppress it. It acts as an intelligent filter, keeping only the most important combinations.

4. nn.Dropout(dropout): The Stress Test If dropout is set to a value like 0.1, this layer randomly deletes 10% of the surviving features during training. This forces the network to distribute its learning and prevents it from relying too heavily on any single feature, making the model more robust. (When actually generating or reconstructing videos during inference, dropout automatically turns itself off).

5. nn.Linear(inner_dim, dim, bias=False): The Executive Summary Now that the features have been expanded, mixed, and aggressively filtered, this final layer compresses the surviving high-quality features back down to the original channel size (dim). This refined, concentrated output is now ready to be passed to the next stage of the network.

Would you like to examine how PEG3D and ffd are wired together inside the full ResNAF block to complete the model's backbone?

User prompt: what does going on. ------ class NAF(nn.Module): def __init__(self, num_layer, dim,): super(NAF, self).__init__() self.num_layer = num_layer self.dconv_layer = nn.Sequential() self.ffd_layer = nn.Sequential() for _ in range(num_layer): self.ffd_layer.append(ffd(dim, 4)) self.dconv_layer.append(PEG3D(dim)) def forward(self, x): for i in range(self.num_layer): x = self.dconv_layer[i](x) x = self.ffd_layer[i](x) return x

Response: The NAF (Neighborhood-Aware Feedforward) class acts as the assembly line that links the two components you just learned (PEG3D and ffd) into a single, cohesive module. It applies a alternating "1-2 punch" to the video data, repeating this process however many times you specify.

The 1-2 Processing Punch

1. **PEG3D (Spatial/Temporal Awareness):** First, the data passes through the depthwise 3D convolution, which looks at the 3D neighborhood (past frames and adjacent pixels) to understand motion and shape without mixing channels.
2. **ffd (Feature Mixing):** Next, the data enters the feedforward network, which looks at each individual position and mixes all of its channels together to build complex features, filtering them through the GEGLU gate.

By pairing them, the model learns *where* things are moving (via PEG3D) and *what* those things are (via ffd) extremely efficiently.

Code Breakdown

- **def __init__(self, num_layer, dim):** Sets up the block. num_layer tells the network how many times to repeat the 1-2 punch, and dim is the number of feature channels.

- **self.dconv_layer = nn.Sequential()** and **self.ffd_layer = nn.Sequential()** create empty containers to hold the sequence of layers.
- **for _ in range(num_layer):** Acts like a factory builder. If num_layer=2, it builds two PEG3D modules and two ffd modules, stacking them in their respective lists.
- **def forward(self, x):** is the actual active assembly line. The for loop takes the input video tensor x and passes it sequentially through the first PEG3D, then the first ffd, then the second PEG3D, then the second ffd, continuing for the total number of layers.

Where This Fits in the Paper In **Figure 1(b)** of the paper, you can see two versions of this block. The standard NAF block (which this code implements, because it lacks residual "skip" connections) is explicitly used inside the **Latent Channel Bottleneck** to help compress and decompress the number of channels using the ISTA-Net+ compressed sensing algorithm. The ResNAF version used in the main encoder/decoder is nearly identical, but it adds the input tensor back to the output at the end of each step.

User prompt: i am going to test this code. let's correct it. ---- NAF_ob = NAF(num_layer=12, dim=512) print(NAF_ob) x = torch.randn(2, 3, 8, 256, 256) output = NAF_ob(x) print(output.shape)

Response: Your test code will throw a shape mismatch error if you run it as written. There are two issues with the input tensor x that need to be corrected:

1. **Channel Mismatch:** You initialized the NAF block with dim=512, which means it expects 512 feature channels. However, your input tensor x only has 3 channels.
2. **Dimension Order:** Standard PyTorch convolutions expect the channel dimension to be second (Batch, Channels, Time, Height, Width). However, if you look at the first line of the PEG3D function you provided earlier, it starts with rearrange(x, 'b t h w d -> b d t h w'). This means the network is specifically coded to expect the channels (d) at the **very end** of the tensor.

Here is the corrected, ready-to-run test code:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from einops import rearrange

# (Assuming PEG3D, GEGLU, ffd, and NAF classes are defined above this)

# Initialize the model
NAF_ob = NAF(num_layer=12, dim=512)
print(NAF_ob)

# Corrected input shape: (Batch, Time, Height, Width, Channels)
# Using smaller spatial dimensions (32x32) so it doesn't crash your computer's
RAM
# when multiplying 512 channels across 12 layers!
x = torch.randn(2, 8, 32, 32, 512)

output = NAF_ob(x)

# The output shape will perfectly match the input shape: [2, 8, 32, 32, 512]
```

```
print(output.shape)
```

A quick tip on memory: A tensor of (2, 8, 256, 256, 512) contains over 536 million floating-point numbers. Passing that through a 12-layer expansion network will instantly cause an Out-Of-Memory (OOM) error on almost any standard GPU. When running local tests to see if the architecture compiles and shapes align, always scale down the height and width (e.g., to 32, 32 or 64, 64) to safely verify the math.

User prompt: RuntimeError: Halving dimension must be even, but dimension 4 is size 1365

Response: The error occurs because the tensor arriving at the GEGLU splitting operation has exactly **1365 channels**, which is an odd number and mathematically cannot be cut in half.

Here is exactly why this happened:

1. **The Math:** Inside your ffd function, when you pass dim=512, the formula inner_dim = int(4 * (2 / 3) * 512) evaluates to exactly **1365**.
2. **The Typo:** In the first nn.Linear layer of your ffd sequence, you likely forgot the * 2 multiplier that was in the original paper's code block.

Check your ffd function definition. You almost certainly wrote this:

```
nn.Linear(dim, inner_dim, bias=False), # Incorrect (outputs 1365 channels)
```

Change it to:

```
nn.Linear(dim, inner_dim * 2, bias=False), # Correct (outputs 2730 channels)
```

Because GEGLU is designed to cut the channel dimension in half (x.chunk(2, dim=-1)), the Linear layer preceding it *must* double the channels first. By restoring the * 2, the Linear layer will output **2730** channels, allowing GEGLU to perfectly slice them into two groups of 1365 without crashing.

---

*User prompt: Traceback (most recent call last): File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/runpy.py", line 196, in _run_module_as_main return _run_code(code, main_globals, None, File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/runpy.py", line 86, in _run_code exec(code, run_globals) File "/home/manish/Desktop/projects/z1-v2/LeanVAE/modules/backbones.py", line 97, in <module> out = NAF_ob(x) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1783, in _wrapped_call_impl return self._call_impl(*args, **kwargs) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1794, in _call_impl return forward_call(*args, **kwargs) File "/home/manish/Desktop/projects/z1-v2/LeanVAE/modules/backbones.py", line 87, in forward x = self.ffd_layer[i](x) File*

*"/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1783, in _wrapped_call_impl return self._call_impl(*args, **kwargs) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1794, in _call_impl return forward_call(*args, **kwargs) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/container.py", line 259, in forward input = module(input) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1783, in _wrapped_call_impl return self._call_impl(*args, **kwargs) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1794, in _call_impl return forward_call(*args, **kwargs) File "/home/manish/Desktop/projects/z1-v2/LeanVAE/modules/backbones.py", line 51, in forward return F.glu(gate) * x File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/functional.py", line 1815, in glu return torch._C._nn.glu(input, dim) RuntimeError: Halving dimension must be even, but dimension 4 is size 1365*

Response: The traceback reveals exactly what went wrong: you accidentally typed F.glu instead of F.gelu inside your custom GEGLU function.

Look closely at this line from your error log: return F.glu(gate) * x

Here is why that specific typo caused the crash:

1. **The Manual Split:** Your code first executes x, gate = x.chunk(2, dim=-1). This correctly takes the 2,730 channels from the previous linear layer and splits them into x (1,365 channels) and gate (1,365 channels).
2. **The F.glu Behavior:** PyTorch's built-in F.glu (Gated Linear Unit) is designed to take a full tensor and **automatically split it in half** internally.
3. **The Crash:** By passing gate to F.glu(gate), you asked PyTorch to split the 1,365 channels in half *again*. Because 1,365 is an odd number, it cannot be divided by 2, triggering the exact Halving dimension must be even error.

To fix this, simply change F.glu back to F.gelu (Gaussian Error Linear Unit). F.gelu acts only as a mathematical activation function and does not attempt to split the tensor.

Corrected GEGLU Function:

```
class GEGLU(nn.Module):
    def forward(self, x):
        x, gate = x.chunk(2, dim=-1)
        return F.gelu(gate) * x  # <-- Changed F.glu back to F.gelu
```

Once you make this single character change, your 12-layer NAF test script will compile and print the correct output shape.

User prompt: why does get this error. --- 2/LeanVAE/modules$ python -m backbones NAF((deconv_layer): Sequential((0): PEG3D((ds_conv): Conv3d(512, 512, kernel_size=(3, 3, 3), stride=(1, 1, 1), groups=512)

*)) (ffd_layer): Sequential((0): Sequential((0): LayerNorm((512,), eps=1e-05, elementwise_affine=True, bias=True) (1): Linear(in_features=512, out_features=2730, bias=False) (2): GEGLU() (3): Dropout(p=0.0, inplace=False) (4): Linear(in_features=2730, out_features=512, bias=False)))) Traceback (most recent call last): File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/runpy.py", line 196, in _run_module_as_main return _run_code(code, main_globals, None, File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/runpy.py", line 86, in _run_code exec(code, run_globals) File "/home/manish/Desktop/projects/z1-v2/LeanVAE/modules/backbones.py", line 120, in <module> out = NAF_ob(x) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1783, in _wrapped_call_impl return self._call_impl(*args, **kwargs) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1794, in _call_impl return forward_call(*args, **kwargs) File "/home/manish/Desktop/projects/z1-v2/LeanVAE/modules/backbones.py", line 87, in forward x = self.ffd_layer[i](x) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1783, in _wrapped_call_impl return self._call_impl(*args, **kwargs) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1794, in _call_impl return forward_call(*args, **kwargs) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/container.py", line 259, in forward input = module(input) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1783, in _wrapped_call_impl return self._call_impl(*args, **kwargs) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/module.py", line 1794, in _call_impl return forward_call(*args, **kwargs) File "/home/manish/anaconda3/envs/cuda121/lib/python3.10/site-packages/torch/nn/modules/linear.py", line 134, in forward return F.linear(input, self.weight, self.bias) RuntimeError: mat1 and mat2 shapes cannot be multiplied (16384x1365 and 2730x512)*

Response: The error occurs because your second Linear layer is expecting 2,730 input channels, but it is only receiving 1,365 channels from the GEGLU layer.

Here is the exact mathematical breakdown of the failure:

1. Your first Linear layer correctly takes the 512 channels and expands them to **2,730**.
2. The GEGLU layer receives those 2,730 channels and splits them perfectly in half (2730÷2=1365), outputting **1,365** channels.
3. The tensor is flattened to a batch size of 16384 (which is 2×8×32×32). This creates your mat1 of size (16384 x 1365).
4. Your second Linear layer's weights (mat2) are sized (2730 x 512). PyTorch cannot multiply a matrix ending in 1,365 with a matrix starting with 2,730.

The Fix: Look closely at the printout for your ffd_layer: (4): Linear(in_features=2730, out_features=512, bias=False)

When fixing the inner_dim * 2 issue from the previous step, you likely accidentally added the * 2 multiplier to the *second* linear layer as well. The second linear layer must take inner_dim (1365), not inner_dim * 2.

Update your ffd function to match this exactly:

```
def ffd(dim, mult=4, dropout=0.):
```

```
    inner_dim = int(mult * (2 / 3) * dim)
    return nn.Sequential(
        nn.LayerNorm(dim),
        nn.Linear(dim, inner_dim * 2, bias=False), # Expands to 2730
        GEGLU(),                                   # Halves to 1365
        nn.Dropout(dropout),
        nn.Linear(inner_dim, dim, bias=False)      # Correct: Takes 1365,
shrinks to 512
    )
```

---

*User prompt: let's teach me this code in deep. ---- def _build_linear_path(self, in_channel=3, out_channel_low=128, out_channel_high=384, pt=2, ph=4, pw=4): patch_config = { 'video_low': (pt, ph, pw), 'video_high': (pt, ph, pw), 'video_low': (1, ph, pw), 'image_hight': (1, ph, pw) } for name, (t, h, w) in patch_config.items(): if 'low' in name: in_dim = in_channel * t * h * w out_dim = out_channel_low else: out_dim = out_channel_high in_dim = in_channel * t * h * w * 7 if 'video' in name else in_channel * t * h * w * 3 proj = nn.Sequential(Rearrange(f'b c (nt {t}) (nh {h}) (nw {w}) -> b nt nh nw (c {t} {h} {w})' if 'video' in name else f'b c (nh {h}) (nw {w}) -> b 1 nh nw (c {h} {w})'), nn.Linear(in_dim, out_dim)) self.add_module(f"{name}_proj", proj)*

Response: _build_linear_path constructs the **Patch Embed** layers inside LeanVAE's **Patchify** module (shown in **Figure 1b** and detailed in **Section 4.1 & 4.2**). After the Haar Wavelet Transform (DWT) splits a video into frequency subbands, this function cuts those subbands into non-overlapping patches and projects them into feature vectors using linear layers (p_{0L},p_{0H},$p_{1:7L}$,$p_{1:7H}$).

⚠️ **Critical Bug Alert in Your Code Snippet** Before diving into the math, look closely at your patch_config dictionary:

```
patch_config = {
    'video_low': (pt, ph, pw),
    'video_high': (pt, ph, pw),
    'video_low': (1, ph, pw),   # <-- BUG 1: Duplicate key! Overwrites
'video_low' and deletes 'image_low'
    'image_hight': (1, ph, pw)  # <-- BUG 2: Typo ('image_hight' instead of
'image_high')
}
```

In Python, having 'video_low' twice overwrites the first (2, 4, 4) entry with (1, 4, 4) and completely fails to create image_low_proj! Make sure to fix those keys in your file to 'image_low' and 'image_high'.

1. Why Are There 4 Separate Paths? (Section 3.1 & 4.2) LeanVAE processes the **first frame** (x_0, treated as a 2D image) separately from the **remaining frames** ($x_{1:T}$, treated as a 3D video) so the model can jointly handle both images and videos:

- **2D Image DWT (First Frame x_0):** A 2D wavelet transform splits 1 RGB image (3 channels) into **4 subbands**:
    - **1 Low-frequency subband (image_low):** 1×3=3 channels.
    - **3 High-frequency subbands (image_high):** 3×3=9 channels (hence the * 3 multiplier in the code).

- **3D Video DWT (Remaining Frames $x_{1:7}$):** A 3D wavelet transform splits the 3D video volume into **8 subbands**:
    - **1 Low-frequency subband (video_low):** 1×3=3 channels.
    - **7 High-frequency subbands (video_high):** 7×3=21 channels (hence the * 7 multiplier in the code).

2. Function Arguments

- **in_channel=3:** The RGB channels of the video.
- **out_channel_low=128, out_channel_high=384:** Matches Section 4.1 of the paper (d_1=128, d_2=384). When concatenated later in the encoder, 128+384=512 (D=512).
- **pt=2, ph=4, pw=4:** The patch size (2×4×4 for video, and 1×4×4 for the first frame). Since the Wavelet Transform already halved the video resolution by 2×2×2, cutting patches of size 2×4×4 achieves the total 4×8×8 compression ratio (2×2=4 in time, 2×4=8 in height/width).

3. Inside the Loop: Calculating in_dim and out_dim To feed a 3D or 2D patch into an nn.Linear layer, you have to flatten all values inside that patch into a single 1D list of numbers (in_dim):

- **video_low:** Takes 1 subband (3 channels) and a patch of 2×4×4. Total numbers in one patch: in_dim = 3×2×4×4=96. It projects 96⟶128 (out_channel_low).
- **image_low:** Takes 1 subband (3 channels) and a patch of 1×4×4. Total numbers: in_dim = 3×1×4×4=48. Projects 48⟶128.
- **video_high:** Takes **7 subbands** (3×7=21 channels) and a patch of 2×4×4. Total numbers: in_dim = 3×2×4×4×7=672. Projects 672⟶384 (out_channel_high).
- **image_high:** Takes **3 subbands** (3×3=9 channels) and a patch of 1×4×4. Total numbers: in_dim = 3×1×4×4×3=144. Projects 144⟶384.

4. Slicing and Projecting (proj = nn.Sequential(...))

- **For Video ('video' in name):** Rearrange('b c (nt 2) (nh 4) (nw 4) -> b nt nh nw (c 2 4 4)') Suppose your DWT output has Time =8, Height =128, Width =128.
    - (nt 2) splits Time (8) into nt=4 chunks of size 2.
    - (nh 4) and (nw 4) split Height and Width (128) into nh=32 and nw=32 chunks of size 4.
    - -> b nt nh nw (c 2 4 4) keeps the grid of patches (b, 4, 32, 32) and flattens every pixel and channel inside each 2×4×4 cube into the final dimension (96 for low, 672 for high).
- **For Image (the else branch):** Rearrange('b c (nh 4) (nw 4) -> b 1 nh nw (c 4 4)') Since the first frame is a 2D image with shape (b, c, h, w), it has no time dimension yet. This slices Height and Width into 4×4 patches, inserts a 1 for the time dimension (b 1 nh nw) so it can later be glued (torch.cat) onto the video patches along the time axis, and flattens (c * 4 * 4) into the last dimension.
- **nn.Linear(in_dim, out_dim):** Takes that flattened patch vector at the end (96, 48, 672, or 144) and compresses/expands it into the target embedding size (128 or 384).
- **self.add_module(f"{name}_proj", proj):** Dynamically registers each of the 4 nn.Sequential pipelines as an official PyTorch layer on self (creating self.video_low_proj, self.video_high_proj, self.image_low_proj, and self.image_high_proj), ensuring their weights are tracked and updated during training.

Would you like to look at how the Wavelet Transform (DWT) feeds into these 4 projections in the forward pass next?

User prompt: let's teach me this code in deep. -------- def _linear_patch(self, x, proj_type): low_comp, high_comp = x[:, :self.in_channel], x[:, self.in_channel:] return getattr(self, f"{proj_type}_low_proj")(low_comp), getattr(self, f"{proj_type}_high_proj")(high_comp) def forward(self, x): xi, xv = x xi_low, xi_high = self._linear_patch(xi, 'image') xv_low, xv_high = self._linear_patch(xv, 'video') low_x = torch.cat([xi_low, xv_low], dim=1) high_x = torch.cat([xi_high, xv_high], dim=1) high_x = self.high_layer(high_x) low_x = self.low_layer(low_x) x = torch.cat([low_x, high_x], dim=-1) x = self.fusion_layer(x) return x def encode(self, x): xi, xv = x if xi is not None and xv is not None: xi_low, xi_high = self._linear_patch(xi, 'image') xv_low, xv_high = self._linear_patch(xv, 'video') low_x = torch.cat([xi_low, xv_low], dim=1) high_x = torch.cat([xi_high, xv_high], dim=1) elif xi is not None: low_x, high_x = self._linear_patch(xi, 'image') elif xv is not None: low_x, high_x = self._linear_patch(xv, 'video') high_x = self.high_layer(high_x) low_x = self.low_layer(low_x) x = torch.cat([low_x, high_x], dim=-1) x = self.fusion_layer(x) return x

Response: This code implements the **Patchify** and **Encoder (ξ)** pipeline of LeanVAE (Specifically **Variant 2**, shown in **Figure 1b**, **Figure 9**, and **Section 4.2** of the paper). It takes the frequency subbands produced by the Haar Wavelet Transform (DWT), slices them into patches, processes low- and high-frequency features separately, and finally fuses them together into the feature tensor p.

1. _linear_patch(self, x, proj_type) — Splitting & Patching Frequencies When the Haar Wavelet Transform (DWT) processes an image or video, it concatenates all frequency subbands along the channel dimension (dim=1), placing the **low-frequency subband (I_{ll} or V_{lll}) first**, followed by the remaining **high-frequency subbands**.

- **low_comp, high_comp = x[:, :self.in_channel], x[:, self.in_channel:]**: Assuming self.in_channel = 3 (RGB), :3 slices out the first 3 channels (the 1 low-frequency subband) as low_comp. 3: grabs all remaining channels as high_comp (9 channels for a 2D image, or 21 channels for a 3D video).
- **getattr(self, f"{proj_type}_low_proj")(low_comp)**: Python's getattr(self, "image_low_proj") is simply a dynamic way of calling self.image_low_proj(low_comp) (the projection layers created in _build_linear_path).
- **Output**: It returns two tensors of shape (Batch, Time_patches, Height_patches, Width_patches, Channels):
    - low patch embedding with 128 channels (d_1=128).
    - high patch embedding with 384 channels (d_2=384).

2. forward(self, x) — The Main Training Pipeline During training, the model always receives a full video clip of T+1 frames (e.g., 17 frames). Before reaching this function, the first frame (x_0) was passed through a 2D DWT (xi), and the remaining T frames ($x_{1:T}$) were passed through a 3D DWT (xv).

- **xi, xv = x**: Unpacks the tuple containing the DWT-transformed first frame (xi) and remaining video frames (xv).
- **xi_low, xi_high = self._linear_patch(xi, 'image')**: Turns the first frame into low-frequency (p_{0L}) and high-frequency (p_{0H}) patches with time length 1.
- **xv_low, xv_high = self._linear_patch(xv, 'video')**: Turns the remaining T frames into low-frequency ($p_{1:TL}$) and high-frequency ($p_{1:TH}$) patches with time length T/4.
- **low_x = torch.cat([xi_low, xv_low], dim=1)** & **high_x = torch.cat([xi_high, xv_high], dim=1)**: Glues the first frame's patches and the video's patches back together along the **Time dimension (dim=1)**. Now both low_x (p_L) and high_x (p_H) have time length $_{4T}$+1.
- **high_x = self.high_layer(high_x)** & **low_x = self.low_layer(low_x)**: In **Equation (5)** and **Figure 1(b)**, the Encoder processes low and high frequencies separately before merging them:

    - self.low_layer (ξ_l): A stack of 2 ResNAF blocks operating on the 128-channel low_x ($\xi_l(p_L)$).
    - self.high_layer (ξ_h): A stack of 2 ResNAF blocks operating on the 384-channel high_x ($\xi_h(p_H)$).
- **x = torch.cat([low_x, high_x], dim=-1)**: Concatenates the processed low-frequency (128) and high-frequency (384) features along the **Channel dimension (dim=-1)**, creating a unified 512-channel tensor (128+384=512).
- **x = self.fusion_layer(x)**: Passes the combined 512-channel tensor through ξ_f (a stack of 4 ResNAF blocks) to fuse the frequency information and output the final encoder feature tensor p. This exact sequence implements **Equation (5)** from the paper:
$p=\xi_f(\text{cat}(\xi_l(p_L),\xi_h(p_H)))$

3. encode(self, x) — Flexible Inference Mode Why is there both forward and encode? Because during inference (testing or Tiling Inference), you might not always have *both* a first frame (xi) and subsequent video frames (xv):

- **Case 1 (xi is not None and xv is not None)**: Standard video encoding (identical to forward).
- **Case 2 (elif xi is not None)**: **Single Image Encoding (T=0)**. If you pass a single 2D image into LeanVAE, there are no following frames (xv is None), or if you are on the very first single-frame step of a custom pipeline. It only runs the 'image' patchifier.
- **Case 3 (elif xv is not None)**: **Temporal Tiling Inference (Chunk 2, 3, ...)**. When encoding a long video in chunks, Chunk 1 contains the very first frame (xi) plus the first few video frames (xv). However, Chunk 2 and onward only contain continuous video frames (xv), so xi is None. This elif branch allows later chunks to bypass the image patchifier and go straight into the video patchifier.

Would you like to trace the exact tensor shapes through forward using a 17-frame 256×256 video, or examine how the ResNAF block or the Compressed Sensing bottleneck (ISTA-Net+) handles the output x next?

User prompt: teach me Decoder_Arch function in deep, as beginner friendly.

Response: Decoder_Arch implements the **Decoder** and **UnPatchify** stages of LeanVAE (described in **Section 4.2**, **Figure 1a**, and **Figure 9**). It acts as the exact mirror image of Encoder_Arch: it takes the reconstructed 512-channel feature tensor p^ from the Compressed Sensing bottleneck (ISTA), splits it back into low- and high-frequency streams, and reshapes those features back into frequency-domain wavelet subbands ready for the Inverse Discrete Wavelet Transform (IDWT).

⚠️ **Critical Bug Alert in Your backbones.py File** Before walking through the logic, look closely at line 255 in your Decoder_Arch._feature_transform method:

```
high_x = self.high_layer(x[:, :, :, :, self.l_dim])  # <-- BUG: Missing colon!
```

Without the colon :, self.l_dim grabs only a **single index** (channel 128) and drops the channel dimension entirely, which will crash self.high_layer with a shape error! Change it to self.l_dim: so it slices all 384 high-frequency channels from index 128 to 512:

```
high_x = self.high_layer(x[:, :, :, :, self.l_dim:]) # <-- FIXED
```

(Also check Encoder_Arch in your file: line 158 is missing a colon in x[:, self.in_channel:], line 159 has parentheses inside getattr(...), and lines 187/189 use if instead of elif!)

1. __init__ — Setting Up the Mirror Architecture Just like the Encoder, the Decoder sets up three ResNAF processing blocks, but it runs them in **reverse order**:

- **self.fusion_layer:** 4 ResNAF layers operating on the combined 512 channels (l_dim + h_dim = 128 + 384).
- **self.low_layer:** 2 ResNAF layers operating on the 128 low-frequency channels (l_dim).
- **self.high_layer:** 2 ResNAF layers operating on the 384 high-frequency channels (h_dim).
- **self._build_linear_unpatch(...):** Builds the 4 linear projection layers that convert feature vectors back into 3D/2D wavelet subband patches.

2. _build_linear_unpatch — Reversing the Patchifier In the Encoder, _build_linear_path flattened patches and compressed them with nn.Linear. Here, _build_linear_unpatch does the exact opposite:

- **Swapped in_dim and out_dim:**
  - **video_low:** Takes in_dim = 128 (out_channel_low) and projects it back to out_dim = 3×2×4×4=96.
  - **image_low:** Projects 128 ⟶ 3×1×4×4=48.
  - **video_high:** Takes in_dim = 384 (out_channel_high) and projects it back to 7 high-frequency subbands: out_dim = 3×2×4×4×7=672.
  - **image_high:** Takes 384 and projects it back to 3 high-frequency subbands: out_dim = 3×1×4×4×3=144.
- **The Unpatching Pipeline (proj = nn.Sequential(...)):**
  1. **nn.Linear(in_dim, out_dim):** Expands the 128 or 384 feature channels into all the raw wavelet coefficients needed for a single patch (96, 48, 672, or 144).
  2. **Rearrange('b nt nh nw (c pt ph pw) -> b c (nt pt) (nh ph) (nw pw)', pt=t, ph=h, pw=w):** Unpacks the flat vector at the end and stitches the patches back together into a full grid:
     - Multiplies the number of temporal patches nt by patch time pt (2 for video, 1 for image).
     - Multiplies height/width patches (nh, nw) by ph=4 and pw=4.
     - Moves the wavelet subband channels c (3 for low, 21 for video_high, 9 for image_high) back to dim=1.

3. _feature_transform(self, x) — Unfusing the Features When the reconstructed tensor *p^* of shape (b, t, h, w, 512) enters the Decoder:

1. **x = self.fusion_layer(x):** First passes the 512-channel tensor through the 4-layer ResNAF fusion block.
2. **low_x = self.low_layer(x[:, :, :, :, :self.l_dim]):** Slices out the first 128 channels (:128) and refines them through the 2-layer low-frequency ResNAF block.
3. **high_x = self.high_layer(x[:, :, :, :, self.l_dim:]):** Slices out the remaining 384 channels (128:512) and refines them through the 2-layer high-frequency ResNAF block.
4. Returns the separated (low_x, high_x) tuple.

4. _linear_unpatch(self, x, proj_type) — Rebuilding Wavelet Subbands

- Receives x = (low_x, high_x) and proj_type ('image' or 'video').
- Passes x[0] (low_x) through image_low_proj or video_low_proj to get the 3-channel low-frequency wavelet subband (low_comp).
- Passes x[1] (high_x) through image_high_proj or video_high_proj to get the 9-channel (image) or

21-channel (video) high-frequency wavelet subbands (high_comp).
- **torch.cat([low_comp, high_comp], dim=1):** Glues them back together along the channel dimension (dim=1), producing a 12-channel tensor for the 2D image (3 + 9) or a 24-channel tensor for the 3D video (3 + 21), ready for the Inverse Wavelet Transform (IDWT).

5. forward(self, x) vs. decode(self, x, is_image=False)

- **forward(self, x) (Standard Training Mode):**
  1. Runs _feature_transform(x) to get low_x and high_x, which both have time length $T'+1$ (e.g., 1+4=5 temporal tokens for a 17-frame video).
  2. **low_x[:, :1], high_x[:, :1]:** Slices out the **first temporal token** (:1, representing the first frame x_0) and unpatches it using 'image' (xi). Notice that for a single image frame, the Rearrange leaves a time dimension of 1 (b c 1 H/2 W/2), which is squeezed or passed into the 2D IDWT.
  3. **low_x[:, 1:], high_x[:, 1:]:** Slices out all **remaining temporal tokens** (1:, representing the video frames $x_{1:T}$) and unpatches them using 'video' (xv).
  4. Returns (xi, xv).
- **decode(self, x, is_image=False) (Flexible Inference Mode):**
  - **If is_image=True:** The input is just a single 2D image (T=0), so all tokens belong to the image. It unpatches the whole tensor with 'image' and returns (xi, None).
  - **If self.tile_inference and not self.is_first_chunk:** During **Tiling Inference** on a long video, Chunk 1 contains the very first image frame (:1) plus video frames (1:), so it uses the else branch. However, **Chunk 2, Chunk 3, etc.** (not self.is_first_chunk) consist *100% of continuous video frames*—there is no first image frame in later chunks! So it sends the entire chunk through 'video' unpatching and returns (None, xv).

Would you like to examine the ISTA class (the Compressed Sensing bottleneck at the bottom of your backbones.py file) next, or write a quick test script to verify Encoder_Arch and Decoder_Arch together?

User prompt: teach me ISTA function in deep, as beginner friendly.

Response: The ISTA class implements the **Channel Compression Bottleneck** of LeanVAE (shown at the bottom of **Figure 1a/b** and explained in **Section 3.2** and **Section 4.2**).

Instead of using two standard linear layers to shrink and expand the channels at the center of the VAE, the authors use a **Compressed Sensing (CS)** algorithm called **ISTA-Net+** (Iterative Shrinkage-Thresholding Algorithm). Its job is to take the rich 512-channel feature tensor p from the Encoder, compress it down into a tiny 4- or 16-channel latent tensor z (for the diffusion model), and then iteratively reconstruct it back into a 512-channel tensor p^ for the Decoder.

1. The Big Idea: Why Compressed Sensing? (Section 3.2) Imagine trying to summarize a 512-page book (p) into a 4-page cheat sheet (z), and then asking someone else to rewrite the full 512-page book (p^) from just that cheat sheet.

- **The Downsampling (Sensing):** You multiply the 512 channels by a "sensing matrix" Φ (self.phi) to get the 4-channel latent $z=\Phi p$.
- **The Upsampling (Iterative Recovery):** Guessing 512 numbers from just 4 numbers in a single step is hard and blurry. Instead, ISTA-Net+ makes an **initial rough guess** (p_0), and then runs a **2-step correction loop (K=2)**. In each loop, it checks: *"If I compress my current 512-channel guess using Φ,*

how far off is it from the true 4-channel latent z?" It uses that error to nudge the guess closer to the truth, filters out noise, and repeats.

2. __init__ — Setting Up the Matrices and Recovery Loop

- **Arguments:**
  - points_num=512: The Encoder's output channel size (*D*=512).
  - out_num=4: The compressed latent channel size (*d*=4 or 16).
  - iter_num=2: The number of iterative refinement loops (*K*=2).
  - layer_num=2: The number of NAF layers inside each forward/backward network.
- **phi_init & self.phi (The Sensing Matrix Φ):**

```
phi_init = np.random.normal(loc=0.0, scale=(1 / points_num) ** 0.5,
size=(out_num, points_num))
self.phi = nn.Parameter(torch.from_numpy(phi_init).float(),
requires_grad=True)
```

  Creates a learnable matrix Φ of shape (4, 512) initialized with small random bell-curve numbers (scaled by $\frac{1}{\sqrt{512}}$ so the variance stays stable). This compresses 512 channels → 4 channels.
- **self.Q (The Initial Reconstruction Matrix Φ~):**

```
self.Q = nn.Parameter(torch.from_numpy(np.transpose(phi_init)).float(),
requires_grad=True)
```

  In **Figure 1(b)** and **Section 4.2**, this is called $\Phi\sim \in R_{D\times d}$. It starts as the transpose of phi_init with shape (512, 4) and is trained to make the first rough guess p_0 by expanding 4 channels → 512 channels.
- **self.forward_l ($F_{(k)}$) & self.backward_l ($F\sim_{(k)}$):** Creates 2 pairs of 2-layer NAF blocks (one pair for each iteration *k*). forward_l transforms the features into a "sparse" space where noise is easy to spot, and backward_l transforms them back.
- **self.weights ($\rho_{(k)}$) & self.threshold ($\theta_{(k)}$):**
  - self.weights: Learnable step sizes $\rho_{(k)}$ (initialized to 1.0) used in **Equation (2)** to control how strongly we correct our guess at each step.
  - self.threshold: Learnable shrinkage thresholds $\theta_{(k)}$ (initialized to 0.01) used in **Equation (3) & (4)** to erase small noisy values close to zero. (Note: self.etas is declared in __init__, but never actually used in the code—it is just a leftover variable from the authors).

3. sample(self, x) — Compressing 512 → 4 Channels This implements the **Sensing Sampling** step ($z=\Phi p$) shown in **Figure 1(b)**:

- **b, t, h, w, d = x.shape:** Suppose x (*p*) has shape (2, 5, 32, 32, 512).
- **y = x.view(-1, d) @ self.phi.T:** Flattens the first four dimensions into a 2D matrix of (10240, 512) and multiplies it (@) by self.phi.T of shape (512, 4). The result y (*z*) has shape (10240, 4).
- **return y.view(b, t, h, w, -1):** Unflattens it back to 5D: (2, 5, 32, 32, 4). This y is your compressed video latent *z*!

4. recon(self, y) — Reconstructing 4 → 512 Channels This implements the **Initial Reconstruction** plus the ***K*-Iteration Loop** from **Figure 1(b)** and **Equations (2), (3), and (4)**:

Step A: Initial Guess ($p_0=\Phi\sim z$)

- **y = y.reshape(-1, c):** Flattens the 4-channel latent *z* to 2D: (10240, 4).
- **recon = torch.mm(y, self.Q.t()):** Multiplies y (10240, 4) by self.Q.t() (4, 512) to produce the initial 512-channel reconstruction p_0 (recon, shape 10240, 512).

Step B: The Iterative Refinement Loop (for i in range(self.iter_num):) Each pass through the loop runs **3 mathematical steps** corresponding directly to the paper's equations:

1. **Gradient Descent Correction — Equation (2): $r_{(k)}=p_{(k-1)}-\rho_{(k)}\Phi^{T}(\Phi p_{(k-1)}-z)$**
   ```
   recon_r = recon - self.weights[i] * torch.mm((torch.mm(recon, self.phi.t())
   - y), self.phi)
   ```
   - torch.mm(recon, self.phi.t()): Compresses our current 512-channel guess (recon) down to 4 channels using Φ.
   - - y: Subtracts the true 4-channel latent y (*z*) to see the **error** between our compressed guess and reality.
   - torch.mm(..., self.phi): Projects that 4-channel error back up to 512 channels using Φ_T.
   - recon - self.weights[i] * ...: Subtracts that scaled error from recon, giving the intermediate corrected feature $r_{(k)}$ (recon_r).
2. **Forward NAF & Soft-Shrinkage Thresholding — Equation (4): $soft(x,\theta)=sgn(x)max(|x|-\theta,0)$**
   ```
   recon = recon_r.reshape(b, t, h, w, -1)
   recon = self.forward_l[i](recon)
   recon = torch.mul(torch.sign(recon), F.relu(torch.abs(recon) -
   self.threshold[i]))
   ```
   - Unflattens recon_r back to 5D (b, t, h, w, 512) so PEG3D inside the NAF block can do 3D convolutions.
   - Passes it through self.forward_l[i] ($F_{(k)}$, a 2-layer NAF block).
   - Soft-Shrinkage (SoftShrink in Fig 1b): F.relu(torch.abs(recon) - self.threshold[i]) takes the absolute value of every number, subtracts the threshold $\theta_{(k)}$ (e.g., 0.01), and turns anything negative into 0. Multiplying by torch.sign(recon) restores the original + or - sign. Simply put: **any weak signal between $-\theta$ and $+\theta$ is wiped to 0**, forcing the features to be clean and sparse.
3. **Backward NAF & Residual Update — Equation (3): $p_{(k)}=r_{(k)}+F\sim_{(k)}(...)$**
   ```
   recon = self.backward_l[i](recon).view(-1, d)
   recon = recon_r + recon
   ```
   - Passes the cleaned, sparse features through self.backward_l[i] ($F\sim_{(k)}$, the second 2-layer NAF block) and flattens it to 2D.
   - Adds the result back to recon_r ($r_{(k)}$) via a skip connection, completing iteration *k*!

Finally, after *K*=2 iterations, return recon.view(b, t, h, w, -1) reshapes the finished 512-channel tensor *p*^ back to (b, t, h, w, 512) so it can enter Decoder_Arch.

Would you like to write a quick test script at the bottom of backbones.py that chains Encoder_Arch → ISTA → Decoder_Arch together to see a complete forward pass in action?

---

*User prompt: what does wrong in this shape. let's correct it. # ([10240, 512]) - 1.0 * {{([10240, 512]), ([512, 4]) -> ([10240, 4]) - ([10240, 512])}} recon_r = recon - self.weights[i] * torch.mm((torch.mm(recon, self.phi.t()) - y), self.phi)*

Response: In your shape comment, two parts are off: y has shape ([10240, 4]) (not 512), and the outer torch.mm(..., self.phi) step that projects the error back to 512 channels is missing from the end of your

comment.

Here is the corrected shape comment:

```
# ([10240, 512]) - 1.0 * {{([10240, 512]) @ ([512, 4]) -> ([10240, 4]) -
([10240, 4])} @ ([4, 512]) -> ([10240, 512])}
recon_r = recon - self.weights[i] * torch.mm((torch.mm(recon, self.phi.t()) -
y), self.phi)
```

Step-by-Step Shape Breakdown

- **torch.mm(recon, self.phi.t())**: Multiplies recon ([10240, 512]) by self.phi.t() ([512, 4]) → outputs ([10240, 4]).
- **- y**: Since y is the compressed 4-channel latent tensor (flattened to ([10240, 4]) at the start of recon), this subtracts ([10240, 4]) - ([10240, 4]) → outputs the 4-channel error ([10240, 4]).
- **torch.mm(..., self.phi)**: Multiplies that 4-channel error ([10240, 4]) by self.phi ([4, 512]) → projects the error back up to ([10240, 512]).
- **recon - self.weights[i] * ...**: Finally, subtracts the scaled error ([10240, 512]) from recon ([10240, 512]) → outputs recon_r with shape ([10240, 512]).

User prompt: what does going on. recon = torch.mul(torch.sign(recon), F.relu(torch.abs(recon) - self.threshold[i]))

Response: This line implements the **Soft-Shrinkage Thresholding** operation (SoftShrink in **Figure 1b** and **Equation 4** of the paper: soft(x,θ)=sgn(x)max($|x|-\theta$,0)). Its job is to act as a **noise gate**: it completely zeroes out any small, weak numbers between $-\theta$ and $+\theta$, and pulls all surviving larger numbers closer to zero by θ.

Step-by-Step Numerical Trace Suppose self.threshold[i] (θ) is **0.01**, and we look at four example values inside recon: **[-0.05, -0.004, 0.008, 0.03]**.

1. **torch.abs(recon) (Make everything positive):** Strips away the negative signs so we only look at the strength (magnitude) of each signal.
   - [-0.05, -0.004, 0.008, 0.03] → **[0.05, 0.004, 0.008, 0.03]**
2. **- self.threshold[i] (Subtract 0.01 from every value):** Any number that was weaker than 0.01 now becomes negative.
   - [0.05, 0.004, 0.008, 0.03] - 0.01 → **[0.04, -0.006, -0.002, 0.02]**
3. **F.relu(...) (Turn all negative numbers into 0):** F.relu(x) is equivalent to max(x,0). This wipes out the weak numbers that dropped below zero!
   - F.relu([0.04, -0.006, -0.002, 0.02]) → **[0.04, 0.0, 0.0, 0.02]**
4. **torch.sign(recon) (Remember the original signs):** Extracts -1 for negative numbers, +1 for positive numbers, and 0 for zero from the original recon.
   - torch.sign([-0.05, -0.004, 0.008, 0.03]) → **[-1, -1, +1, +1]**
5. **torch.mul(...) (Restore the original signs):** Multiplies the signos back onto the filtered magnitudes:
   - [-1, -1, +1, +1] * [0.04, 0.0, 0.0, 0.02] → **[-0.04, 0.0, 0.0, 0.02]**

Notice what happened: the two weak noisy numbers (-0.004 and 0.008) were erased to **0.0**, while the two strong signals (-0.05 and 0.03) survived (shrunk slightly by 0.01 to -0.04 and 0.02). In Compressed

Sensing, this enforces **sparsity** (Equation 1, $// \Psi p //_1$), ensuring only strong, meaningful video features are passed to the backward NAF network.

User prompt: what does going on. # [-0.04, 0.0, 0.0, 0.02] -> recon = self.backward_l[i](recon).view(-1, d)

Response: This line executes the **backward transformation ($F\sim_{(k)}$)** from **Equation (3)** and **Figure 1(b)** of the paper, taking the cleaned, noise-filtered features and translating them back into the main feature space so they can be added to recon_r.

1. Why Do We Need self.backward_l[i] ($F\sim_{(k)}$)? Think of the three steps in Equation (3) like cleaning a noisy audio recording:

- **Step 1 (self.forward_l[i] / $F_{(k)}$):** Transforms the video features into a specialized "sparse" domain where true signal and background noise are separated.
- **Step 2 (SoftShrink):** Mutes the quiet background noise to 0.0 (producing [-0.04, 0.0, 0.0, 0.02]).
- **Step 3 (self.backward_l[i] / $F\sim_{(k)}$):** Now that the noise is erased, self.backward_l[i] (which is a 2-layer NAF block) translates those surviving clean numbers ([-0.04, 0.0, 0.0, 0.02]) **back into the original 512-channel representation space**. Because NAF mixes neighboring pixels (PEG3D) and channels (ffd), those 0.0 slots don't stay zero—they get filled in with clean, reconstructed feature refinements calculated from the strong surviving signals (-0.04 and 0.02).

2. Step-by-Step Shape & Data Trace

- **Input recon:** Enters with 5D shape ([2, 5, 32, 32, 512]) containing the thresholded values [-0.04, 0.0, 0.0, 0.02, ...].
- **self.backward_l[i](recon):** Passes the 5D tensor through the 2-layer NAF network (PEG3D → ffd → PEG3D → ffd), outputting a dense, refined 5D tensor of the exact same shape ([2, 5, 32, 32, 512]) (e.g., [-0.031, 0.012, -0.005, 0.019, ...]).
- **.view(-1, d):** Flattens the first four dimensions (b, t, h, w) back into a 2D matrix of shape **([10240, 512])** (since 2×5×32×32=10240 and d = 512).

3. Why Flatten with .view(-1, d) at the End? Look at the very next line in your code:

```
recon = recon_r + recon
```

Earlier in the loop, recon_r was calculated as a 2D matrix of shape ([10240, 512]). By flattening the output of self.backward_l[i] back to ([10240, 512]) using .view(-1, d), PyTorch can directly add recon_r + recon together to complete **Equation (3)** ($p_{(k)}=r_{(k)}+F\sim_{(k)}(...)$) and stay in 2D for the start of the next iteration.

User prompt: teach me Patcher function i deep, as beginner friedly.

Response: Patcher implements the **Haar Discrete Wavelet Transform (DWT)** described in **Section 3.1**, **Section 4.2**, and **Appendix B (Figure 6)** of the LeanVAE paper. Before the video ever touches a neural network layer (Encoder_Arch), Patcher uses fixed mathematical filters to split the raw RGB video into **low-frequency subbands** (smooth shapes/averages) and **high-frequency subbands** (sharp edges and

fast motion), while cutting the resolution in half.

⚠️ **Critical Bug Alert in Your patches.py File** Look closely at line 70 inside _3ddwt:

```
xlhh = F.conv3d(xll, hh.unsqueeze(2).unsqueeze(3), groups=g, stride=(1, 1, 2))
# <-- BUG: Passes xll instead of xlh!
```

Notice that xlhh is supposed to apply the high-pass filter (hh) to **xlh**, not xll! Because you passed xll, xlhh is an exact duplicate of xllh (line 68) and completely throws away the xlhh frequency subband. Change xll to xlh on line 70:

```
xlhh = F.conv3d(xlh, hh.unsqueeze(2).unsqueeze(3), groups=g, stride=(1, 1, 2))
# <-- FIXED
```

1. The Core Idea: How Does a Haar Wavelet Work? Imagine two neighboring pixels with brightness values [A, B]. The Haar wavelet simply calculates two things:

1. **Low-Pass Filter (hl — The Average):** Adds them together: $\frac{1}{\sqrt{2}}A+\frac{1}{\sqrt{2}}B$. This captures the smooth, blurry average of the two pixels.
2. **High-Pass Filter (hh — The Difference):** Subtracts them: $\frac{1}{\sqrt{2}}A-\frac{1}{\sqrt{2}}B$. If two neighboring pixels are identical, their difference is 0. If there is a sharp edge or sudden motion between them, the difference is large! This captures fine details and edges.

Why use 0.70710678 ($\frac{1}{\sqrt{2}}$) instead of 0.5? In math, dividing by 2 makes the transform **orthonormal** (energy-preserving), meaning the Inverse Wavelet Transform (IDWT) can later reverse the process with zero loss of information.

2. __init__ — Setting Up the Fixed Math Constants

- **self.register_buffer("wavelets", torch.tensor([0.70710678..., 0.70710678...]))**: Registers the two $\frac{1}{\sqrt{2}}$ numbers as a PyTorch "buffer". A buffer is a tensor that automatically moves to the GPU when you call .cuda(), but is **not** a trainable weight.
- **self.register_buffer("_arange", torch.arange(2))**: Stores [0, 1]. Why? Because $(-1)^{[0,1]}=[1,-1]$. Multiplying wavelets by [1, -1] turns the [+, +] low-pass filter into the [+, -] high-pass filter!
- **for param in self.parameters(): param.requires_grad = False**: Ensures nothing in this class is ever modified during backpropagation.

3. _2ddwt — 2D Wavelet Transform for the First Frame (x_0) In **Section 3.1**, a 2D image $I \in \mathbb{R}^{c \times h \times w}$ is decomposed along Width and Height into **4 subbands**: $\{I_{ll}, I_{lh}, I_{hl}, I_{hh}\}$, each of size $c \times \frac{h}{2} \times \frac{w}{2}$.

- **x = x.squeeze(2):** The first frame arrives with 5D shape (B, 3, 1, H, W). .squeeze(2) removes the 1 in the time dimension so it becomes a standard 4D image (B, 3, H, W) for F.conv2d.
- **Building hl and hh:**
  - hl: [0.7071, 0.7071] repeated for all g=3 RGB channels ⟶ shape (3, 1, 2).
  - hh: [0.7071, -0.7071] repeated for all g=3 RGB channels ⟶ shape (3, 1, 2).
- **x = F.pad(x, pad=(n-2, n-1, n-2, n-1), mode=mode):** Since n = 2 (the filter length), pad=(0, 1, 0, 1) adds 1 reflected pixel to the right and bottom edges so even/odd dimensions divide cleanly.
- **Pass 1 — Filtering Along Width (stride=(1, 2)):** hl.unsqueeze(2) makes the filter horizontal (1x2).

With stride=(1, 2), the filter steps by **2 pixels horizontally**, cutting the Width in half ($W \to W/2$):
    - xl: Horizontal low-pass → shape (B, 3, H, W/2).
    - xh: Horizontal high-pass → shape (B, 3, H, W/2).
- **Pass 2 — Filtering Along Height (stride=(2, 1)):** hl.unsqueeze(3) makes the filter vertical (2x1). With stride=(2, 1), it steps by **2 pixels vertically**, cutting the Height in half ($H \to H/2$):
    - xll: Low-Width + Low-Height (the blurry thumbnail I_{ll}).
    - xlh, xhl, xhh: The 3 high-frequency edge subbands.
- **out = torch.cat([xll, xlh, xhl, xhh], dim=1):** Glues all 4 subbands along the channel dimension (dim=1), turning 3 channels into **4×3=12 channels**.
- **if rescale: out = out * 2:** Because we multiplied by $\frac{1}{2}$ twice (once for Width, once for Height), the average pixel value got scaled by $\frac{1}{2} \times \frac{1}{2} = \frac{1}{2}$. Multiplying by 2 restores the pixel brightness back to the normal scale.
- **return out.unsqueeze(2):** Puts the time dimension of 1 back in, returning shape (B, 12, 1, H/2, W/2).

4. _3ddwt — 3D Wavelet Transform for Video Frames ($x_{1:7}$) In **Section 3.1** and **Appendix B (Figure 6)**, a 3D video $V \in \mathbb{R}_{c \times t \times h \times w}$ is filtered across **Time, Height, and Width**, splitting into **8 subbands** (2×2×2=8): $\{V_{lll}, V_{llh}, V_{lhl}, V_{lhh}, V_{hll}, V_{hlh}, V_{hhl}, V_{hhh}\}$, each of size $c \times \frac{t}{2} \times \frac{h}{2} \times \frac{w}{2}$:

- **Pass 1 — Time Axis (stride=(2, 1, 1)):** Filters adjacent frames in time (hl.unsqueeze(3).unsqueeze(4) has shape 2x1x1), cutting Time in half ($T \to T/2$) to produce xl (slow temporal average) and xh (fast temporal motion).
- **Pass 2 — Height Axis (stride=(1, 2, 1)):** Filters xl and xh vertically (1x2x1), cutting Height in half ($H \to H/2$) to produce 4 intermediate tensors: xll, xlh, xhl, xhh.
- **Pass 3 — Width Axis (stride=(1, 1, 2)):** Filters those 4 tensors horizontally (1x1x2), cutting Width in half ($W \to W/2$) to produce all **8 subbands** (xlll through xhhh).
- **out = torch.cat([xlll, ..., xhhh], dim=1):** Concatenates all 8 subbands along the channel dimension (dim=1), turning 3 RGB channels into **8×3=24 channels** (the first 3 are low-frequency xlll, and the remaining 21 are high-frequency).
- **if rescale: out = out * (2 * torch.sqrt(torch.tensor(2.0))):** Because we applied the $\frac{1}{2}$ filter 3 times (Time, Height, Width), $(\frac{1}{2})^3 = \frac{1}{2\sqrt{2}}$. Multiplying by $2\sqrt{2}$ normalizes the scale.
- **Output Shape:** (B, 24, T/2, H/2, W/2).

5. forward(self, x) — Routing Images, Videos, and Tiled Chunks The input x enters with shape (Batch, 3, Frames, Height, Width):

- **Case 1: Standard Video (x.shape[2] > 1 and x.shape[2] % 2 == 1 — e.g., 17 frames)**
    - xi, xv = torch.split(x, [1, x.shape[2] - 1], dim=2): Splits the 17 frames along Time (dim=2) into the **first frame xi (1 frame)** and **remaining frames xv (16 frames)**.
    - Runs _2ddwt(xi) → shape (B, 12, 1, H/2, W/2).
    - Runs _3ddwt(xv) → shape (B, 24, 8, H/2, W/2).
    - Returns (xi, xv), which goes straight into Encoder_Arch.forward!
- **Case 2: Even Number of Frames (x.shape[2] > 1 and else — e.g., 4 or 8 frames in Tiling Inference Chunk 2+)**
    - When processing subsequent chunks during Tiling Inference, there is no "first image frame" (x_0), so the chunk has an even number of continuous video frames. It runs _3ddwt(x) on the entire chunk and returns (None, xv).
- **Case 3: Single 2D Image (x.shape[2] == 1)**
    - If x has only 1 frame, it skips the if x.shape[2] > 1 block, runs _2ddwt(x), and returns (xi, None).

Would you like to look at the UnPatcher class (the Inverse Discrete Wavelet Transform IDWT that reverses

this process after Decoder_Arch) next?

User prompt: i was use MAGVIT paper, in 40 gb a100 17 frame is working, video shape=([2, 3, 17, 256, 256]) when i use LeanVAE can i use 33 frame on this gpu memory. MAGVIT paper=LANGUAGE MODEL BEATS DIFFUSION — TOKENIZER IS KEY TO VISUAL GENERATION

Response: Yes, you can easily fit a ([2, 3, 33, 256, 256]) video on a 40GB A100 with LeanVAE—for both inference and full end-to-end training—with plenty of VRAM to spare.

1. Why MAGVIT Hits the 40GB Ceiling at 17 Frames MAGVIT-v2 (*Language Model Beats Diffusion*) relies on a heavy 3D CNN hierarchy that starts processing at the full 17 x 256 x 256 resolution and progressively downsamples through dense 3D ResNet blocks. Because standard 3D convolutions mix all channels across 3×3×3 volumes at high spatial resolutions (256x256 and 128x128), intermediate activation maps for backprop consume tens of gigabytes of VRAM.

2. Why LeanVAE Uses a Fraction of the Memory LeanVAE eliminates high-resolution 3D convolutions entirely at the front of the network:

- **Immediate 4×8×8 Compression Before the Backbone:** First, the parameter-free Haar Wavelet Transform (Patcher) halves the dimensions, and the linear patchifier immediately slices the rest by 2×4×4.
- **Tiny Tensor Shape Inside the Network:** For your ([2, 3, 33, 256, 256]) input (*T*=32), the tensor entering the ResNAF Encoder, ISTA bottleneck, and ResNAF Decoder has already shrunk to:
    - **Time:** $_{432}$+1=9 temporal tokens
    - **Height & Width:** $_{8256}$=32×32 spatial tokens
    - **Backbone Tensor Shape:** ([2, 9, 32, 32, 512])
- **Lightweight Depthwise Convolutions:** Every ResNAF and NAF block only runs on this tiny 9 x 32 x 32 grid using depthwise 3D convolutions (groups=dim).

3. Concrete VRAM Numbers on Your 40GB A100

- **Inference:** In Table 4 of the paper, LeanVAE processes a 17 x 256 x 256 video using just **2.3 GB** of VRAM, and even a 17 x 1920 x 1080 (1080p) video takes only **15 GB** in FP16. Doubling a 256x256 clip to 33 frames with batch size 2 will use under **4–5 GB** of VRAM.
- **Training (with VGG LPIPS + 3D PatchGAN):** Notice in Section 5.1 that the authors trained LeanVAE on 8× 48GB GPUs with a total batch size of 40 (**batch size 5 per GPU**) at resolutions up to 352x288. At ([2, 3, 33, 256, 256]), the LeanVAE backbone activations and weights (40M parameters) take only a few gigabytes. Your largest memory consumer during training won't even be LeanVAE itself—it will be computing the VGG-16 perceptual loss (LPIPS) across all 2×33=66 frames. Even with LPIPS and the PatchGAN discriminator active, a batch size of 2 at 33 frames (256x256) will comfortably fit within **~14–18 GB** in bf16/fp16 (or ~24–28 GB in fp32), well inside your 40GB A100 limit.

If you ever scale up to 65+ frames and find the VGG LPIPS loss eating too much memory, you can either run torch.amp.autocast('cuda', dtype=torch.bfloat16) or compute LPIPS on a random subset of frames per step. Are you planning to train LeanVAE from scratch on 33-frame clips, or use it frozen to extract latents for your diffusion model?

---

*User prompt: in this code. there is no any multi-gpu handle process, rank and world_size. ---- import os import argparse import torch import pytorch_lightning as pl from pytorch_lightning.callbacks import ModelCheckpoint, LearningRateMonitor from pytorch_lightning.loggers import WandbLogger from LeanVAE import VideoData, LeanVAE, AutoEncoderEngine from LeanVAE.utils.callbacks import VideoLogger def main(): pl.seed_everything(1234) parser = argparse.ArgumentParser() parser.add_argument('--pretrained', type=str, default=None) parser.add_argument('--ckpt_path', type=str, default='') parser.add_argument('--bf16', action='store_true') parser.add_argument('--fp16', action='store_true') parser = pl.Trainer.add_argparse_args(parser) parser = VideoData.add_data_specific_args(parser) parser = LeanVAE.add_model_specific_args(parser) parser = AutoEncoderEngine.add_model_specific_args(parser) args = parser.parse_args() data = VideoData(args) model = AutoEncoderEngine(args, data) if args.pretrained is not None: load_weights = torch.load(args.pretrained, map_location='cpu')["state_dict"] msg = model.load_state_dict(load_weights, strict=False) missing_keys = msg.missing_keys unexpec_keys = msg.unexpected_keys print(f"Model loaded from {args.pretrained}.") print(f"Missing: {missing_keys}") print(f"Unexpected: {unexpec_keys}") callbacks = [] callbacks.append(ModelCheckpoint(monitor='val/recon_loss', save_top_k=3, mode='min', filename='latest_checkpoint')) callbacks.append(ModelCheckpoint(every_n_train_steps=3000, save_top_k=-1, filename='{epoch}-{step}-{recon_loss:.2f}')) callbacks.append(LearningRateMonitor(logging_interval='step')) print("Log the reconstructed videos...") callbacks.append(VideoLogger(batch_frequency=1500, max_videos=4, clamp=True)) kwargs = dict() if args.gpus > 1: kwargs = dict(accelerator='gpu', gpus=args.gpus) if args.bf16: kwargs = dict(accelerator='gpu', gpus=args.gpus, precision="bf16") if args.fp16: kwargs = dict(accelerator='gpu', gpus=args.gpus, precision=16) wandb_logger = WandbLogger(project="LeanVAE", name=os.path.basename(args.default_root_dir), save_dir=args.default_root_dir, config=args) trainer = pl.Trainer.from_argparse_args(args, log_every_n_steps=49, logger=wandb_logger, callbacks=callbacks, replace_sampler_ddp=False, limit_val_batches=0, num_sanity_val_steps=0, max_steps=args.max_steps, **kwargs) trainer.fit(model, data, ckpt_path = args.ckpt_path if hasattr(args, 'ckpt_path') else '') if __name__ == '__main__': main()*

Response: Multi-GPU training, rank, and world_size are actually fully supported in this script, because **PyTorch Lightning (pytorch_lightning as pl)** handles all the Distributed Data Parallel (DDP) boilerplate automatically behind the scenes.

In raw PyTorch, you have to manually write dist.init_process_group(backend="nccl"), pass rank and world_size, wrap the model in DistributedDataParallel(model, device_ids=[local_rank]), and attach a DistributedSampler to your DataLoader. In PyTorch Lightning, pl.Trainer does all of that internally the moment you set --gpus greater than 1.

1. Where Multi-GPU is Configured in This Code

- **parser = pl.Trainer.add_argparse_args(parser):** This single line injects all of PyTorch Lightning's built-in distributed training command-line flags (--gpus, --num_nodes, --strategy, --accelerator) directly into your argparse parser.
- **if args.gpus > 1::** When you pass --gpus 2 or --gpus 8 in your terminal, this block sets accelerator='gpu' and gpus=args.gpus.
- **trainer = pl.Trainer.from_argparse_args(..., **kwargs):** When Trainer sees gpus > 1, it automatically spawns one Python process per GPU, sets LOCAL_RANK, RANK, and WORLD_SIZE environment variables, initializes NCCL, wraps AutoEncoderEngine in DDP, and averages gradients across all GPUs during trainer.fit().

2. Crucial Detail to Watch Out For: replace_sampler_ddp=False Look closely at this argument inside pl.Trainer.from_argparse_args(...):

```
replace_sampler_ddp=False
```

Normally, PyTorch Lightning automatically injects a DistributedSampler into your DataLoader so each GPU gets a different slice of the dataset (rank and world_size). Because the authors set replace_sampler_ddp=False, **Lightning will not add a sampler automatically**—it expects VideoData (LeanVAE/data.py) to manually create the DistributedSampler (or shard an IterableDataset using trainer.global_rank and trainer.world_size). If VideoData doesn't have a DistributedSampler inside it, every GPU will train on the exact same batch of videos unless you change replace_sampler_ddp=True.

3. Explicitly Setting the ddp Strategy (Recommended Fix) In older versions of PyTorch Lightning (v1.5–1.9, which uses add_argparse_args and gpus=), passing accelerator='gpu', gpus=args.gpus without specifying a strategy can sometimes default to dp (DataParallel on a single process) or throw a deprecation warning depending on your installed version. To ensure true multi-process DDP is used, add strategy='ddp' inside the if args.gpus > 1: block:

```
    kwargs = dict()
    if args.gpus > 1:
        kwargs = dict(accelerator='gpu', gpus=args.gpus, strategy='ddp')
        if args.bf16:
            kwargs = dict(accelerator='gpu', gpus=args.gpus, strategy='ddp',
precision="bf16")
        if args.fp16:
            kwargs = dict(accelerator='gpu', gpus=args.gpus, strategy='ddp',
precision=16)
```

*(Note: If you also want --bf16 or --fp16 to work when testing on a **single GPU** (--gpus 1), move the if args.bf16: and if args.fp16: checks outside of if args.gpus > 1:, because right now mixed precision is ignored when args.gpus == 1!)*

4. How to Access rank and world_size Inside the Model If you ever need to check the current GPU's rank or world_size inside AutoEncoderEngine or VideoData, PyTorch Lightning attaches them directly to the module:

- **self.global_rank:** The global rank of the process (0 to world_size - 1).
- **self.local_rank:** The GPU index on the current machine (0 to gpus - 1).
- **self.trainer.world_size:** Total number of GPUs across all nodes.

To launch multi-GPU training from your terminal (for example, on 4 GPUs with bf16), you simply run:

```
python train.py --gpus 4 --bf16 --default_root_dir ./logs/leanvae_run
```

Would you like to inspect your VideoData class (data.py) to verify whether it properly shards the dataset

across rank and world_size when replace_sampler_ddp=False?

User prompt: let's update the code, i use new version of pytorch_lighning.

Response: In PyTorch Lightning 2.0+, several legacy methods (pl.Trainer.add_argparse_args, pl.Trainer.from_argparse_args, gpus=, and replace_sampler_ddp) were removed.

Here is the updated training script compatible with modern PyTorch Lightning (v2.x), including explicit Trainer arguments, updated precision strings ("bf16-mixed", "16-mixed"), and proper DDP configuration:

```
import os
import argparse
import torch
import pytorch_lightning as pl
from pytorch_lightning.callbacks import ModelCheckpoint, LearningRateMonitor
from pytorch_lightning.loggers import WandbLogger
from LeanVAE import VideoData, LeanVAE, AutoEncoderEngine
from LeanVAE.utils.callbacks import VideoLogger

def main():
    pl.seed_everything(1234, workers=True)

    parser = argparse.ArgumentParser()
    # Custom checkpoint & precision arguments
    parser.add_argument('--pretrained', type=str, default=None)
    parser.add_argument('--ckpt_path', type=str, default=None)
    parser.add_argument('--bf16', action='store_true')
    parser.add_argument('--fp16', action='store_true')

    # Trainer arguments (replaces removed pl.Trainer.add_argparse_args)
    parser.add_argument('--gpus', type=int, default=1, help='Number of GPUs to
use')
    parser.add_argument('--num_nodes', type=int, default=1, help='Number of
nodes for distributed training')
    parser.add_argument('--max_steps', type=int, default=600000, help='Maximum
training steps')
    parser.add_argument('--default_root_dir', type=str, default='./logs',
help='Directory for logs and checkpoints')
    parser.add_argument('--accumulate_grad_batches', type=int, default=1)
    parser.add_argument('--gradient_clip_val', type=float, default=None)

    # Module-specific arguments
    parser = VideoData.add_data_specific_args(parser)
    parser = LeanVAE.add_model_specific_args(parser)
    parser = AutoEncoderEngine.add_model_specific_args(parser)

    args = parser.parse_args()

    data = VideoData(args)
```

```
    model = AutoEncoderEngine(args, data)

    if args.pretrained is not None:
        load_weights = torch.load(args.pretrained,
map_location='cpu')["state_dict"]
        msg = model.load_state_dict(load_weights, strict=False)
        print(f"Model loaded from {args.pretrained}.")
        print(f"Missing: {msg.missing_keys}")
        print(f"Unexpected: {msg.unexpected_keys}")

    callbacks = [
        ModelCheckpoint(
            monitor='val/recon_loss',
            save_top_k=3,
            mode='min',
            filename='latest_checkpoint'
        ),
        ModelCheckpoint(
            every_n_train_steps=3000,
            save_top_k=-1,
            filename='{epoch}-{step}-{recon_loss:.2f}'
        ),
        LearningRateMonitor(logging_interval='step'),
        VideoLogger(batch_frequency=1500, max_videos=4, clamp=True)
    ]
    print("Log the reconstructed videos...")

    # Precision setup (works for both single-GPU and multi-GPU in PL 2.x)
    precision = "32-true"
    if args.bf16:
        precision = "bf16-mixed"
    elif args.fp16:
        precision = "16-mixed"

    # Strategy setup for Multi-GPU DDP
    # Note: If GAN discriminator parameters are unused in the first 600k steps,
    # use "ddp_find_unused_parameters_true" instead of "ddp"
    strategy = "ddp_find_unused_parameters_true" if args.gpus > 1 else "auto"

    wandb_logger = WandbLogger(
        project="LeanVAE",
        name=os.path.basename(os.path.normpath(args.default_root_dir)),
        save_dir=args.default_root_dir,
        config=vars(args)
    )

    trainer = pl.Trainer(
        accelerator="gpu",
        devices=args.gpus,
        num_nodes=args.num_nodes,
        strategy=strategy,
        precision=precision,
```

```
        default_root_dir=args.default_root_dir,
        max_steps=args.max_steps,
        accumulate_grad_batches=args.accumulate_grad_batches,
        gradient_clip_val=args.gradient_clip_val,
        log_every_n_steps=49,
        logger=wandb_logger,
        callbacks=callbacks,
        use_distributed_sampler=False,  # Replaces replace_sampler_ddp=False
        limit_val_batches=0,
        num_sanity_val_steps=0,
    )

    ckpt_path = args.ckpt_path if args.ckpt_path else None
    trainer.fit(model, datamodule=data, ckpt_path=ckpt_path)


if __name__ == '__main__':
    main()
```

Key Updates Made for PyTorch Lightning 2.x

- **Removed add_argparse_args & from_argparse_args:** Lightning 2.0 removed automatic argparse injection; the required flags (--gpus, --max_steps, --default_root_dir) are now explicitly defined and passed to pl.Trainer(...).
- **gpus= ⟶ accelerator="gpu", devices=args.gpus:** Replaces the removed gpus keyword argument.
- **replace_sampler_ddp=False ⟶ use_distributed_sampler=False:** Renamed in Lightning 2.0. *(If your VideoData class does not manually create a DistributedSampler, change this to use_distributed_sampler=True so multi-GPU ranks don't duplicate batches).*
- **Updated Precision Strings:** Changed "bf16" and 16 to "bf16-mixed" and "16-mixed", and moved them outside the args.gpus > 1 check so mixed precision works on 1 GPU as well.
- **strategy="ddp_find_unused_parameters_true":** During the first 600k steps of LeanVAE training, the GAN discriminator weight is 0 (λ_{adv}=0), or the generator and discriminator alternate steps. Standard "ddp" will crash if a module has unused parameters during a step; "ddp_find_unused_parameters_true" prevents that crash.

⚠️ **One Important Check in AutoEncoderEngine (pl.LightningModule)** If AutoEncoderEngine has **two optimizers** (one for the VAE and one for the PatchGAN discriminator) and took an optimizer_idx argument in training_step(self, batch, batch_idx, optimizer_idx) under Lightning 1.x, note that Lightning 2.0 removed optimizer_idx in favor of manual optimization (self.automatic_optimization = False). If you share your AutoEncoderEngine code next, we can check if its training_step needs that quick update as well.

---

*User prompt: teach me this function code, what does meaning of this function, and why does need this. -------- def get_length_grouped_indices(lengths, batch_size, world_size, gradient_accumulation_size, initial_global_step, generator=None, group_data=False, seed=42): # We need to use torch for the random part as a distributed sampler will set the random seed for torch. if generator is None: generator = torch.Generator().manual_seed(seed) # every rank will generate a fixed order but random index # print('lengths', lengths) if group_data: indices = group_data_fun(lengths, generator) else: indices = torch.randperm(len(lengths), generator=generator).tolist() megabatch_size = world_size **

*batch_size megabatches = [indices[i: i + megabatch_size] for i in range(0, len(lengths), megabatch_size)] megabatches_len = [[lengths[i] for i in megabatch] for megabatch in megabatches] megabatches = [split_to_even_chunks(megabatch, lengths, world_size, batch_size) for megabatch in megabatches] split_to_even_chunks_len = [[[lengths[i] for i in batch] for batch in megabatch] for megabatch in megabatches] indices_mega = torch.randperm(len(megabatches), generator=generator).tolist() # print(f'rank {accelerator.process_index} seed {seed}, len(megabatches) {len(megabatches)}, indices_mega, {indices_mega[:50]}') shuffled_megabatches = [megabatches[i] for i in indices_mega] shuffled_megabatches_len = [[[lengths[i] for i in batch] for batch in megabatch] for megabatch in shuffled_megabatches] # print(f'\nrank {accelerator.process_index} sorted shuffled_megabatches_len', shuffled_megabatches_len[0], shuffled_megabatches_len[1], shuffled_megabatches_len[-2], shuffled_megabatches_len[-1]) # import ipdb;ipdb.set_trace() # print('shuffled_megabatches', len(shuffled_megabatches)) if group_data: shuffled_megabatches = last_group_data_fun(shuffled_megabatches, lengths) group_shuffled_megabatches_len = [[[lengths[i] for i in batch] for batch in megabatch] for megabatch in shuffled_megabatches] # print(f'\nrank {accelerator.process_index} group_shuffled_megabatches_len', group_shuffled_megabatches_len[0], group_shuffled_megabatches_len[1], group_shuffled_megabatches_len[-2], group_shuffled_megabatches_len[-1]) #initial_global_step = initial_global_step * gradient_accumulation_size #todo shuffled_megabatches = shuffled_megabatches[initial_global_step:] #print(f'Skip the data of {initial_global_step} step!') return [batch for megabatch in shuffled_megabatches for batch in megabatch]*

Response: get_length_grouped_indices is a **smart distributed batch builder** used inside your VideoData sampler. Remember how the training script had replace_sampler_ddp=False (or use_distributed_sampler=False)? This function is the exact reason why: it builds a custom, synchronized list of video indices across all GPUs (world_size) so that **videos of similar resolutions or frame lengths are grouped into the same training step**.

1. Why Do We Need This Function? (The 3 Big Problems It Solves) In LeanVAE (Section 5.1), the model is trained on videos of *varying resolutions* (ranging from 96×128 to 352×288) and mixed images/videos. If you use a standard random DataLoader, three bad things happen:

- **Problem 1: Shape Mismatch Inside a Batch:** PyTorch cannot stack a 96×128 video and a 352×288 video into the same batch tensor x without padding or cropping them to the same size, which wastes memory and compute.
- **Problem 2: Multi-GPU Straggler Bottleneck (DDP Sync):** Suppose you train on 4 GPUs (world_size = 4). In Distributed Data Parallel (DDP), all 4 GPUs must finish their forward and backward pass before they can average their gradients and move to the next step. If GPU 0 gets a giant 352×288 video batch (takes 0.5 seconds) and GPU 1 gets a tiny 96×128 video batch (takes 0.1 seconds), **GPU 1 sits completely idle for 0.4 seconds** waiting for GPU 0!
- **Problem 3: Resuming from a Checkpoint:** If training crashes at step 50,000, a normal DataLoader starts over from step 0 of the epoch. This function slices off the first 50,000 steps (shuffled_megabatches[initial_global_step:]) so training resumes on the exact next unseen batch.

2. Key Vocabulary in This Code

- **lengths:** A list containing a "size number" for every video in your dataset (e.g., [17, 17, 1, 33, ...] or a combined resolution/frame code). If you have 1,000 videos, len(lengths) == 1000.
- **batch_size:** How many videos **one single GPU** processes at once (e.g., 2).
- **world_size:** Total number of GPUs (e.g., 4).
- **megabatch:** The total number of videos processed across **all GPUs combined in a single training step**:

megabatch_size=world_size×batch_size
For example, with 4 GPUs and batch_size = 2, one megabatch holds 4×2=8 videos.

3. Line-by-Line Code Breakdown

Step 1: Synchronize Randomness Across All GPUs

```
if generator is None:
    generator = torch.Generator().manual_seed(seed)
```

In multi-GPU DDP, every GPU runs its own separate Python process. By fixing manual_seed(seed) (e.g., 42), **every single GPU generates the exact same sequence of random numbers** without needing to talk to each other over the network.

Step 2: Group Similar Videos Together (group_data)

```
if group_data:
    indices = group_data_fun(lengths, generator)
else:
    indices = torch.randperm(len(lengths), generator=generator).tolist()
```

- If group_data=False, torch.randperm just shuffles the dataset indices (0 to N-1) completely randomly.
- If group_data=True, group_data_fun sorts/buckets the dataset indices so videos with the same length (resolution/frame count) sit next to each other in the indices list.

Step 3: Slice Into "Megabatches" (One Megabatch per Training Step)

```
megabatch_size = world_size * batch_size
megabatches = [indices[i: i + megabatch_size] for i in range(0, len(lengths),
megabatch_size)]
```

Slices the flat list of indices into chunks of size megabatch_size.

- *Example:* If world_size = 2 and batch_size = 2, megabatch_size = 4.
- megabatches looks like: [[id1, id2, id3, id4], [id5, id6, id7, id8], ...]. Each inner list of 4 videos represents **1 global training step**.

Step 4: Balance the Workload Across GPUs (split_to_even_chunks)

```
megabatches = [split_to_even_chunks(megabatch, lengths, world_size, batch_size)
for megabatch in megabatches]
```

Inside each megabatch (e.g., 4 videos), split_to_even_chunks divides the 4 videos into world_size (2) mini-batches of size batch_size (2)—one for GPU 0, and one for GPU 1—so that both GPUs get an **equal computational load** based on lengths.

- Now megabatches is a 3D list: [[[gpu0_vid1, gpu0_vid2], [gpu1_vid1, gpu1_vid2]], ...].

- *(Note: megabatches_len and split_to_even_chunks_len are just debug variables created for the commented-out print statements—they don't affect the output at all).*

Step 5: Shuffle the Order of the Megabatches

```
indices_mega = torch.randperm(len(megabatches), generator=generator).tolist()
shuffled_megabatches = [megabatches[i] for i in indices_mega]
```

Why shuffle *again*? Because in Step 2, group_data_fun sorted the videos by size! If we didn't shuffle the megabatches, the model would train ONLY on small videos for the first hour, and ONLY on large videos for the last hour, which ruins gradient descent. Shuffling the megabatches keeps **every video inside a step the same size**, while making the **sequence of steps random** (Step 1 is 256×256, Step 2 is 96×128, Step 3 is 352×288, etc.).

Step 6: Handle Leftover/Boundary Groups (last_group_data_fun)

```
if group_data:
    shuffled_megabatches = last_group_data_fun(shuffled_megabatches, lengths)
```

Cleans up any edge-case megabatches (like the very last incomplete batch of a specific video resolution) so mixed-resolution batches don't accidentally cause a shape crash.

Step 7: Fast-Forward When Resuming a Checkpoint (initial_global_step)

```
shuffled_megabatches = shuffled_megabatches[initial_global_step:]
```

If you start a brand-new training run, initial_global_step is 0, so it keeps all megabatches. If you resume from a checkpoint at step 15000, it slices [15000:], instantly skipping the first 15,000 megabatches you already trained on!

Step 8: Flatten and Return

```
return [batch for megabatch in shuffled_megabatches for batch in megabatch]
```

Unpacks the 3D list [megabatch -> gpu_batch -> video_ids] into a 2D list of batches: [[step0_gpu0_batch], [step0_gpu1_batch], [step1_gpu0_batch], [step1_gpu1_batch], ...]. Later in the sampler, GPU 0 will grab indices 0, 2, 4, ... and GPU 1 will grab indices 1, 3, 5, ...!

Would you like to look at split_to_even_chunks or the Sampler class that calls get_length_grouped_indices to see how each GPU picks its own batch using rank?

User prompt: training and validating both are working once time. there are lot's of there found val code.

Response: In the pl.Trainer setup from your training script, validation is actually **turned off** right now because of limit_val_batches=0, even though AutoEncoderEngine and VideoData contain full validation code and ModelCheckpoint(monitor='val/recon_loss', ...) tries to watch val/recon_loss.

1. Why Validation Wasn't Running (limit_val_batches=0) Look at these two arguments inside your pl.Trainer(...) call:

```
limit_val_batches=0,
num_sanity_val_steps=0,
```

- **limit_val_batches=0:** Explicitly tells PyTorch Lightning to process **0 validation batches**—meaning validation_step inside AutoEncoderEngine is completely skipped during trainer.fit(), and ModelCheckpoint(monitor='val/recon_loss') never receives a score to save latest_checkpoint.
- **num_sanity_val_steps=0:** Skips the 2 "test validation steps" that PyTorch Lightning normally runs at the very beginning before step 1 of training.

2. How Training and Validation Work Together in PyTorch Lightning In a large video dataset like Kinetics-600, a single epoch can take tens of thousands of steps. If you wait for a full training epoch to finish before validating, you won't see validation results for days. Instead, you configure pl.Trainer to pause training every *N* steps, run the validation loop (val_dataloader → validation_step), save the best checkpoint (val/recon_loss), and immediately resume training:

Update your pl.Trainer(...) in train.py with these 4 validation settings:

```
    trainer = pl.Trainer(
        accelerator="gpu",
        devices=args.gpus,
        num_nodes=args.num_nodes,
        strategy=strategy,
        precision=precision,
        default_root_dir=args.default_root_dir,
        max_steps=args.max_steps,
        accumulate_grad_batches=args.accumulate_grad_batches,
        gradient_clip_val=args.gradient_clip_val,
        log_every_n_steps=49,
        logger=wandb_logger,
        callbacks=callbacks,
        use_distributed_sampler=False,
        # --- ENABLE STEP-BASED VALIDATION ---
        limit_val_batches=1.0,        # 1.0 = run 100% of val_dataloader (or
set e.g. 50 to run 50 batches)
        val_check_interval=3000,      # Run validation every 3,000 training
steps
        check_val_every_n_epoch=None, # Required in PL 2.x when
val_check_interval > batches in 1 epoch
        num_sanity_val_steps=2,       # Run 2 val batches at start to catch any
val bugs immediately
    )
```

3. What Each Setting Does

- **limit_val_batches=1.0:** Enables the validation loop (1.0 runs the full validation set; if your validation set is huge and slows down training, you can set an integer like limit_val_batches=50 to only evaluate 50 batches per check).
- **val_check_interval=3000:** Tells Lightning to run validation every 3000 training steps (matching your every_n_train_steps=3000 checkpoint callback).
- **check_val_every_n_epoch=None:** Tells Lightning to trigger validation strictly by **step count** (3000 steps) rather than waiting for epoch boundaries.
- **num_sanity_val_steps=2:** Runs 2 quick validation batches before step 0 of training. This is super helpful because if there is a bug in your validation code or val_dataloader, it fails in the first 10 seconds instead of crashing after 3,000 training steps!

Would you like to share the validation code from VideoData (val_dataloader) or AutoEncoderEngine (validation_step) so we can walk through how it works or check it for PyTorch Lightning 2.x compatibility?

User prompt: what does mean of this code. ------- def val_dataloader(self): return self.video_data._dataloader(False)[0]

Response: val_dataloader is a built-in hook required by **PyTorch Lightning**. Whenever the Trainer pauses training to run a validation check, it calls this exact method to get the DataLoader that feeds validation videos into your model's validation_step.

Breakdown of return self.video_data._dataloader(False)[0]

- **self.video_data:** References the VideoData instance attached to your module (data = VideoData(args)).
- **._dataloader(False):** Calls the internal helper method _dataloader(is_train) inside VideoData. Passing False tells it **"this is NOT for training"**—so it loads the **validation split** of your video dataset instead of the training split, turns off random training augmentations/shuffling, and builds the validation DataLoader.
- **[0]:** The _dataloader(...) helper method returns a tuple or a list (typically (dataloader, sampler) or [dataloader]). Adding [0] grabs just the first item—the actual PyTorch DataLoader object—because PyTorch Lightning's val_dataloader() expects a DataLoader to be returned, not a tuple.

(Note: By comparison, train_dataloader(self) in the same file likely calls self.video_data._dataloader(True)[0], passing True to load the training split).

---

*User prompt: what does meaning of this function. ------- def save_video_grid(video, fname, nrow=None, fps=3): b, c, t, h, w = video.shape video = video.permute(0, 2, 3, 4, 1).contiguous() video = (video.detach().cpu().numpy() * 255).astype('uint8') if nrow is None: nrow = math.ceil(math.sqrt(b)) ncol = math.ceil(b / nrow) padding = 0 video_grid = np.zeros((t, (padding + h) * nrow + padding, (padding + w) * ncol + padding, c), dtype='uint8') # print(video_grid.shape) for i in range(b): r = i // ncol c = i % ncol start_r = (padding + h) * r start_c = (padding + w) * c video_grid[:, start_r:start_r + h, start_c:start_c + w] = video[i] video = [] for i in range(t): video.append(video_grid[i]) imageio.mimsave(fname, video, fps=fps) # skvideo.io.vwrite(fname,*

video_grid, inputdict={'-r': '5'}) # print('saved videos to', fname)

Response: save_video_grid takes a batch of multiple PyTorch video tensors (b videos), stitches them together side-by-side into a single visual grid (like a CCTV monitor screen or Zoom gallery view), and saves that combined animation to your hard drive as an .mp4 or .gif file. This is typically called by the VideoLogger callback during training so you can visually inspect the original input videos next to LeanVAE's reconstructed outputs.

Line-by-Line Code Breakdown

1. Prepare the Tensor for Image Viewing

- **b, c, t, h, w = video.shape:** Unpacks the 5 dimensions of the batch: Batch size (b), RGB Channels (c=3), Time frames (t), Height (h), and Width (w).
- **video = video.permute(0, 2, 3, 4, 1).contiguous():** PyTorch models keep RGB channels in the 2nd slot (b, c, t, h, w), but video-saving libraries require RGB channels at the very end. .permute(0, 2, 3, 4, 1) reorders the dimensions to **(b, t, h, w, c)**.
- **video = (video.detach().cpu().numpy() * 255).astype('uint8'):**
    - .detach().cpu().numpy(): Disconnects the tensor from PyTorch's gradient tracker, moves it from the GPU to standard system RAM (CPU), and converts it into a NumPy array.
    - * 255 and .astype('uint8'): Neural networks work with floating-point numbers between 0.0 and 1.0. Standard video files require whole integers between 0 and 255 (uint8 = 8-bit unsigned integer).

2. Calculate the Grid Layout (nrow and ncol)

- **if nrow is None: nrow = math.ceil(math.sqrt(b)):** If you don't specify how many rows you want in the grid, it automatically picks a square-like number using the square root of the batch size b. For example, if b = 4 videos, 4=2 rows (nrow = 2).
- **ncol = math.ceil(b / nrow):** Calculates how many columns are needed to fit all b videos. For b = 4 and nrow = 2, ncol = 4 / 2 = 2 columns (creating a 2×2 grid).

3. Create the Blank Canvas (video_grid)

- **padding = 0:** Sets the border gap between videos to 0 pixels.
- **video_grid = np.zeros((t, (padding + h) * nrow + padding, (padding + w) * ncol + padding, c), dtype='uint8'):** Creates a giant, black (all-zeros) video canvas big enough to hold all b videos tiled together.
    - For example, if you have 4 videos of size 17 frames x 256 height x 256 width in a 2×2 grid, video_grid has shape **(17, 512, 512, 3)**.

4. Paste Each Video Into Its Grid Slot (for i in range(b):)

- **r = i // ncol** and **c = i % ncol:** Uses integer division (//) and modulo remainder (%) to find the exact Row (r) and Column (c) coordinate for video i:
    - Video 0: Row 0 // 2 = 0, Col 0 % 2 = 0 (Top-Left)
    - Video 1: Row 1 // 2 = 0, Col 1 % 2 = 1 (Top-Right)
    - Video 2: Row 2 // 2 = 1, Col 2 % 2 = 0 (Bottom-Left)
    - Video 3: Row 3 // 2 = 1, Col 3 % 2 = 1 (Bottom-Right)
- **start_r = (padding + h) * r** and **start_c = (padding + w) * c:** Converts the row/column index into

exact pixel coordinates (e.g., Row 1 starts at pixel 256).

- **video_grid[:, start_r:start_r + h, start_c:start_c + w] = video[i]:** Copies all t frames of video i into its assigned pixel box on the video_grid canvas. *(Note: Notice that inside this loop, the variable c for column overwrites the earlier c variable for channels, which is safe here only because c for channels isn't used again after np.zeros).*

5. Save the Final Video File

- **video = [] / for i in range(t): video.append(video_grid[i]):** Splits the 4D video_grid array (t, H_grid, W_grid, 3) into a Python list of t individual 2D RGB frames, each of shape (H_grid, W_grid, 3).
- **imageio.mimsave(fname, video, fps=fps):** Uses the imageio library to encode that list of frames into a video file at path fname playing at fps frames per second (default 3 frames per second, which plays the video in slow motion so you can easily spot reconstruction artifacts frame-by-frame).

Would you like to look at the VideoLogger callback that calls save_video_grid, or inspect AutoEncoderEngine next?

User prompt: what does meaning of this function. ----- def log_vid(self, pl_module, batch, batch_idx, split="train"): # print(batch_idx, self.batch_freq, self.check_frequency(batch_idx) and hasattr(pl_module, "log_videos") and callable(pl_module.log_videos) and self.max_videos > 0) if (self.check_frequency(batch_idx) and # batch_idx % self.batch_freq == 0 hasattr(pl_module, "log_videos") and callable(pl_module.log_videos) and self.max_videos > 0): # print(batch_idx, self.batch_freq, self.check_frequency(batch_idx)) logger = type(pl_module.logger) is_train = pl_module.training if is_train: pl_module.eval() with torch.no_grad(): videos = pl_module.log_videos(batch, split=split, batch_idx=batch_idx) for k in videos: N = min(videos[k].shape[0], self.max_videos) videos[k] = videos[k][:N] if isinstance(videos[k], torch.Tensor): videos[k] = videos[k].detach().cpu() if self.clamp: videos[k] = torch.clamp(videos[k], -0.5, 0.5) self.log_local(pl_module.logger.save_dir, split, videos, pl_module.global_step, pl_module.current_epoch, batch_idx) if is_train: pl_module.train()

Response: log_vid is the core method inside the VideoLogger callback (which you initialized in train.py with VideoLogger(batch_frequency=1500, max_videos=4, clamp=True)). Every 1,500 batches during training, it temporarily pauses training, generates reconstructed videos from the current batch, cleans up the tensors, and saves them to disk (calling save_video_grid from the previous step) so you can visually monitor how well LeanVAE is learning.

Line-by-Line Code Breakdown

1. The Gatekeeper Check (if (...))

```
if (self.check_frequency(batch_idx) and
        hasattr(pl_module, "log_videos") and
        callable(pl_module.log_videos) and
        self.max_videos > 0):
```

Before doing any work, it checks four conditions:

- **self.check_frequency(batch_idx):** Checks if batch_idx % 1500 == 0. Since saving videos to disk is slow, we only want to do it once every 1,500 batches, not on every single step.
- **hasattr(pl_module, "log_videos") & callable(...):** Verifies that your PyTorch Lightning model (pl_module, which is AutoEncoderEngine) actually has a function named log_videos(...) inside it.
- **self.max_videos > 0:** Ensures you didn't disable video logging by setting max_videos=0.

2. Safely Switch to Evaluation Mode

```
is_train = pl_module.training
if is_train:
    pl_module.eval()
```

- **is_train = pl_module.training:** Remembers whether the model was currently in training mode (True).
- **pl_module.eval():** Temporarily switches the model to evaluation mode so layers like Dropout are turned off while generating the sample videos.

3. Generate the Reconstructed Videos Without Using Extra GPU Memory

```
with torch.no_grad():
    videos = pl_module.log_videos(batch, split=split, batch_idx=batch_idx)
```

- **with torch.no_grad():** Turns off PyTorch's gradient tracking engine so generating these preview videos doesn't eat up your 40GB A100 VRAM.
- **videos = pl_module.log_videos(...):** Calls the log_videos method inside AutoEncoderEngine, which runs the current batch through LeanVAE and returns a Python dictionary videos containing both the original ground-truth videos and the reconstructed videos (for example, {"inputs": x, "reconstructions": x_recon}).

4. Trim, Move to CPU, and Clamp Pixel Values (for k in videos:)

```
for k in videos:
    N = min(videos[k].shape[0], self.max_videos)
    videos[k] = videos[k][:N]
    if isinstance(videos[k], torch.Tensor):
        videos[k] = videos[k].detach().cpu()
        if self.clamp:
            videos[k] = torch.clamp(videos[k], -0.5, 0.5)
```

Loops over each entry k in the videos dictionary (e.g., "inputs" and "reconstructions"):

- **N = min(videos[k].shape[0], self.max_videos) & videos[k][:N]:** If your batch size is 8 and self.max_videos = 4, this slices [:4] so it only saves the first 4 videos.
- **.detach().cpu():** Moves the video tensor off the GPU onto CPU system RAM so it can be converted to NumPy.
- **torch.clamp(videos[k], -0.5, 0.5):** In LeanVAE's data pipeline, RGB pixel values are normalized to the range **[-0.5, 0.5]** (centered around 0.0). Early in training, the model might output wild numbers like -1.2 or +2.3. torch.clamp forces any value below -0.5 to -0.5 and any value above 0.5 to 0.5 so the colors don't overflow or glitch when saved. *(Inside self.log_local, the code will add + 0.5 to shift [-0.5, 0.5] back to [0.0, 1.0] before calling save_video_grid).*

5. Save to Disk and Resume Training

```
self.log_local(pl_module.logger.save_dir, split, videos,
               pl_module.global_step, pl_module.current_epoch, batch_idx)

if is_train:
    pl_module.train()
```

- **self.log_local(...):** Passes the cleaned video dictionary, current training step (global_step), and folder path (save_dir) to log_local, which formats the filenames and calls save_video_grid to write the .mp4/.gif files to your log folder.
- **if is_train: pl_module.train():** Switches the model back into training mode (.train()) so your training loop can continue seamlessly on the next step.

Would you like to check log_local or AutoEncoderEngine's training_step / log_videos methods next?
